ASE CHAPTER 7

T1 Multiple-Choice Questions Pack

60 MCQs

Section 1: Official Course-Slide MCQs (50 Items)

Q7.1. Which one is not a phase of "bath tub curve" of hardware reliability?

(a) Burn-in · (b) Useful life · (c) Wear-out · (d) Test-out

(d) Test-out

The classical hardware reliability "bathtub curve" depicts failure rate over time across three distinct operational phases:

1. **Burn-in phase (Infant mortality): Characterized by an initially high failure rate that decreases rapidly as manufacturing defects and marginal components are identified and replaced.**
2. **Useful life phase: Characterized by a low, relatively constant failure rate driven by random, unpredictable environmental stresses.**
3. **Wear-out phase: Characterized by a steadily increasing failure rate caused by physical deterioration, fatigue, oxidation, and mechanical friction.**

"Test-out" is not a recognized operational phase of the bathtub curve.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 308; companion slides 3, 108.

📝 **Academic Note**

ملاحظة الطالب والفخ الامتحاني:

- المراحل الثلاث لمنحنى الحوض (Bathtub curve) في عتاد الأجهزة هي: مرحلة البداية (Burn-in)، ثم العمر النافع المستقر (Useful life)، ثم مرحلة الاهتراء والتقادم (Wear-out).
- لا توجد مرحلة باسم «Test-out»، فالبرمجيات لا تهترئ مادياً (No wear-out) بعكس الهاردوير.

Q7.2. Software reliability is:

(a) The probability of failure free operation of a program for a specified time in a specified environment · (b) The probability of failure of a program for a specified time in a specified environment · (c) The probability of success of a program for a specified time in any environment · (d) None of the above

(a) The probability of failure free operation of a program for a specified time in a specified environment

Software reliability is the probability of failure-free operation of a program for a specified time in a specified environment.

The definition specifies four dimensions:

- **Probability:** A mathematical quantification bounded in $[0, 1]$.
- **Failure-free operation:** The absence of dynamic non-conformances with user expectations.
- **Specified time:** Measured in execution (CPU) time, clock time, or operational units.
- **Specified environment:** The input space and operational profile under which the software executes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 308–309; companion slides 6, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- احفظ أركان تعريف اعتمادية البرمجيات الأربعة: الاحتمالية (Probability) ← التشغيل الخالي من الفشل (Failure-free) ← في زمن محدد (Specified time) ← في بيئة تشغيلية محددة (Specified environment).

Q7.3. Fault is:

(a) Defect in the program · (b) Mistake in the program · (c) Error in the program · (d) All of the above

(d) All of the above

In IEEE standard terminology and software reliability foundations, a fault is an internal manifestation of a human mistake or error that introduces a defect into the program source code, design, or documentation. While rigorous taxonomies distinguish the developer's cognitive error (mistake) from the static code defect (fault) and the runtime event (failure), the terms defect, mistake, and error are commonly used interchangeably in introductory reliability literature to characterize a fault.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 310; companion slides 8, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تسلسل الأعطال القياسي: الخطأ البشري (Error/Mistake) يُنتج عيباً داخلياً في الكود يسمى العلة أو الخلل (Fault/Defect)، والذي يسبب عند تنفيذه الفشل التشغيلي الخارجي (Failure).

Q7.4. One fault may lead to:

(a) One failure · (b) Two failures · (c) Many failures · (d) All of the above

(d) All of the above

There is no strict one-to-one mapping between static faults and dynamic failures:

- **A single fault may manifest as a single failure** under a specific test case.
- **A single fault may trigger multiple recurring failures** across different user transactions or continuous iterations.
- **A single fault may cause an entire cascade of failures** in downstream distributed components.
- Conversely, **multiple distinct faults may combine to trigger a single observable failure.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 310; companion slides 8, 108.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العلاقة بين العيب المصدري (Fault) والفشل التشغيلي (Failure) هي علاقة (One-to-Many)؛ عيب برمجي واحد في حلقة تكرار قد يسبب آلاف الانهيارات، وقد لا يظهر أبداً إذا لم يُنفذ مساره.

Q7.5. Which "time" unit is not used in reliability studies?

(a) Execution time · (b) Machine time · (c) Clock time · (d) Calendar time

(b) Machine time

Software reliability engineering characterizes system behavior across three specific, well-defined time concepts:

1. **Execution time (τ): The actual processor (CPU) time spent by the computer system executing the instructions of the software under study.**
2. **Calendar time (t): The ordinary chronological time experienced in calendar days, weeks, or months**, including idle time and off-shift hours.
3. **Clock time: The total elapsed wall-clock running time during which the system is operational.**

"Machine time" is an ambiguous, non-standard term not formally recognized as a time dimension in reliability modeling.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 310–311; companion slides 11, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المقاييس الزمنية المعتمدة في دراسات الاعتمادية هي ثلاثة: زمن التنفيذ الفعلي للمعالج (τ Execution time)، والزمن التقويمي الطبيعي (t Calendar time)، وزمن الساعة الجدارية (Clock time). مصطلح «Machine time» مستبعد.

Q7.6. Failure occurrences can be represented as:

(a) Time to failure · (b) Time interval between failures · (c) Failures experienced in a time interval · (d) All of the above

(d) All of the above

In stochastic reliability modeling, the random failure process can be mathematically formulated in three mathematically equivalent representations:

1. **Time to failure:** The elapsed time from the start of execution until failure occurrence.
2. **Time interval between failures:** The incremental operational duration separating successive failure events ($t_i - t_{i-1}$).
3. **Failures experienced in a time interval:** The cumulative or incremental count of failures observed over a given operational duration ($M(\tau)$ or $\Delta\mu$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 311; companion slides 12, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يمكن تمثيل حدوث الانهيارات بثلاث طرق إحصائية متكافئة: زمن حدوث الفشل، أو الفاصل الزمني بين فشلين، أو عدد الإخفاقات خلال فترة محددة.

Q7.7. Maximum possible value of reliability is:

(a) 100 · (b) 10 · (c) 1 · (d) 0

(c) 1

Because reliability $R(t)$ is mathematically defined as a probability, its values are strictly bounded by the probability axioms within the closed real interval:

$$0 \leq R(t) \leq 1$$

A reliability value of $R(t) = 1.0$ represents a 100% mathematical probability of failure-free operation over the specified interval. (While percentages such as 100% are colloquial, the raw mathematical maximum is 1).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 7, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الاعتمادية هي دالة احتمالية رياضية (Probability)، وبالتالي فإن قيمتها الصغرى هي 0 وقيمتها العظمى هي 1 (ولا يُختار 100 إلا إذا وُضعت علامة النسبة المئوية %).

Q7.8. Minimum possible value of reliability is:

(a) 100 · (b) 10 · (c) 1 · (d) 0

(d) 0

The lower bound of the probability distribution for software reliability is 0, representing complete operational failure where failure-free operation over the specified duration cannot occur.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 7, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- القيمة الصغرى المطلقة للاعتمادية هي 0.

Q7.9. As the reliability increases, failure intensity:

(a) Decreases · (b) Increases · (c) No effect · (d) None of the above

(a) Decreases

Reliability $R(t)$ and failure intensity $\lambda(t)$ share an inverse exponential relationship:

$$R(t) = \exp\left(-\int_0^t \lambda(u)\,du\right)$$

When failure intensity λ decreases (indicating fewer failures occurring per unit of execution time), the probability of failure-free operation $R(t)$ increases toward 1. Conversely, high failure intensity rapidly drives reliability toward zero.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 312, 321; companion slides 21, 109.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العلاقة عكسية صريحة: كلما انخفضت شدة الفشل (Failure Intensity λ)، ارتفعت موثوقية واعتمادية النظام (R).

Q7.10. If failure intensity is 0.005 failures/hour during 10 hours of operation of a software, its reliability can be expressed as:

(a) 0.10 · (b) 0.92 · (c) 0.95 · (d) 0.98

(c) 0.95

Under the constant failure intensity assumption over operational duration t, reliability is calculated using the exponential reliability formula:

$$R(t) = e^{-\lambda t}$$

Substituting $\lambda = 0.005$ failures/hour and $t = 10$ hours:

$$R(10) = e^{-(0.005 \times 10)} = e^{-0.05} \approx 0.951229 \approx 0.95$$

Therefore, the probability of failure-free operation during the 10-hour interval is approximately 0.95 (or 95.1%).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 22, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- القانون الرياضي المباشر: $R = e^{-\lambda t} = e^{-(0.005 \times 10)} = e^{-0.05} \approx 0.9512$. التقريب الصحيح هو 0.95.

Q7.11. Software Quality is:

(a) Conformance to requirements · (b) Fitness for the purpose · (c) Level of satisfaction · (d) All of the above

(d) All of the above

Software quality is a multidimensional concept synthesizing several canonical engineering perspectives:

- **Crosby's definition:** "Conformance to requirements" (meeting specified functional and non-functional specifications).
- **Juran's definition:** "Fitness for purpose / use" (satisfying customer operational needs in real production).
- **IEEE / ISO definition:** The degree to which a software system satisfies user requirements and produces a high level of stakeholder satisfaction.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 321; companion slides 24, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- جودة البرمجيات تشمل التوافق مع المتطلبات المحددة (Crosby)، والملاءمة للغرض التشغيلي (Juran)، وتحقيق رضا المستخدم (ISO/IEEE).

Q7.12. Defect rate is:

(a) Number of defects per million lines of source code · (b) Number of defects per function point · (c) Number of defects per unit of size of software · (d) All of the above

(d) All of the above

Defect rate (or defect density) is defined as the total number of identified defects normalized against a standardized measure of software size. Industry and academic standards compute defect rate per KLOC (thousands of lines of code), per million lines of code (in telecommunications/avionics), or per Function Point (in business IT systems). Therefore, any metric normalizing defects per unit of size represents defect rate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 322; companion slides 27, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معدل أو كثافة العيوب (Defect rate) هو مقياس منسوب لوحدة الحجم، سواء قيس بعدد العيوب لكل KLOC أو لكل Function Point.

Q7.13. How many product quality factors have been proposed in McCall quality model?

(a) 2 · (b) 3 · (c) 11 · (d) 6

(c) 11

In Jim McCall's seminal software quality model (1977), software quality is decomposed into exactly 11 product quality factors organized across three operational perspectives:

1. **Product Operation (5 factors):** Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision (3 factors):** Maintainability, Flexibility, Testability.
3. **Product Transition (3 factors):** Portability, Reusability, Interoperability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 323–324; companion slides 30–35, 110.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- نموذج ماكول (McCall) يقسم الجودة إلى **3 أبعاد رئيسية** تضم بداخلها **11 عاملاً نوعياً (Factors)**: 5 للتشغيل + 3 للمراجعة والتعديل + 3 للانتقال وقابلية النقل.

Q7.14. Which one is not a product quality factor category of McCall quality model?

(a) Product revision · (b) Product operation · (c) Product specification · (d) Product transition

(c) Product specification

McCall's triangular quality framework categorizes its 11 factors into three broad operational categories:

- **Product Operation:** Using the software to perform real-world tasks.
- **Product Revision:** Modifying and debugging the software code.
- **Product Transition:** Porting and adapting the software to new platforms.

"Product specification" is an engineering phase, not one of McCall's three quality categories.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- أبعاد ماكول الثلاثة الشهيرة هي: Product Operation وProduct Revision وProduct Transition. لا يوجد بعد باسم «Product specification».

Q7.15. The second level of quality attributes in McCall quality model are termed as:

(a) Quality criteria · (b) Quality factors · (c) Quality guidelines · (d) Quality specifications

(a) Quality criteria

McCall's quality model is structured in a strict hierarchical decomposition:

1. **Top Level (High-level Factors):** External quality factors representing management and customer viewpoints (e.g., Maintainability, Reliability).
2. **Second Level (Quality Criteria): Intermediate software attributes representing software engineering viewpoints that directly affect the factors** (e.g., modularity, conciseness, consistency, fault tolerance).
3. **Third Level (Quality Metrics):** Quantitative direct measurements used to calculate the criteria.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 323–324; companion slides 31, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الهرمية في نموذج ماكول: العوامل (Factors) في القمة ← تليها المعايير الهندسية (Criteria) في المستوى الثاني ← تليها المقاييس الكمية المباشرة (Metrics).

Q7.16. Which one is not a level in Boehm software quality model?

(a) Primary uses · (b) Intermediate constructs · (c) Primitive constructs · (d) Final constructs

(d) Final constructs

Barry Boehm's hierarchical quality model (1978) structures quality across three explicit levels:

1. **Primary uses / high-level characteristics:** The figure links General Utility directly to Portability and Maintainability. As-is Utility links to Reliability, Efficiency, and Human Engineering; Maintainability links to Testability, Understandability, and Modifiability.
2. **Intermediate constructs:** Seven constructs, including Portability, Reliability, Efficiency, Human Engineering, Testability, Understandability, and Modifiability.
3. **Primitive constructs:** The model lists 12 lower-level characteristics.

"Final constructs" is not a level in Boehm's taxonomy.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 324–326; companion slides 40, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مستويات نموذج بويم (Boehm) الثلاثة هي: Primary uses ← تليها Intermediate constructs ← ثم Primitive constructs. لا توجد طبقة تسمى «Final constructs».

Q7.17. Which one is not a software product quality model?

(a) McCall model · (b) Boehm model · (c) ISO 9000 · (d) ISO 9126

(c) ISO 9000

McCall's model, Boehm's model, and ISO 9126 are product quality models that evaluate the internal attributes and external runtime characteristics of the software artifact itself.

In contrast, ISO 9000 (and ISO 9001) is a Quality Management System (QMS) standard focused strictly on organizational development processes, management commitment, and procedural compliance, rather than evaluating software product attributes directly.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 321, 354–355; companion slides 29, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فرق جوهري: نماذج جودة المنتج البرمجي تشمل McCall وBoehm وISO 9126. أما ISO 9000 فهو معيار إدارة عمليات المنشأة (Process Standard) وليس مقياساً لجودة المنتج البرمجي ذاته.

Q7.18. Basic execution time model was developed by:

(a) Bev. Littlewood · (b) J.D. Musa · (c) Barry Boehm · (d) Jim McCall

(b) J.D. Musa

The Basic Execution Time Model was formulated and published by John D. Musa at Bell Laboratories in 1979. It represents one of the foundational Non-Homogeneous Poisson Process (NHPP) software reliability growth models based on execution time.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 329; companion slides 47, 111.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- نموذج زمن التنفيذ الأساسي (Basic Execution Time Model) وضعه جون موسى (J.D. Musa) في مختبرات بيل عام 1979.

Q7.19. NHPP stands for:

(a) Non Homogeneous Poisson Process · (b) Non Heterogeneous Poisson Process · (c) Non Homogeneous Poisson Product · (d) Non Heterogeneous Poisson Product

(a) Non Homogeneous Poisson Process

NHPP stands for Non-Homogeneous Poisson Process. In software reliability growth modeling, an NHPP is a generalized Poisson stochastic process whose failure rate (intensity function $\lambda(t)$) varies dynamically over execution time rather than remaining constant.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 329; companion slides 47, 112.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح NHPP يعني عملية بواسون غير المتجانسة (Non-Homogeneous Poisson Process)، وتتميز بأن معدل حدوث الأخطاء فيها يتغير مع تقدم زمن الاختبار.

Q7.20. In the Basic execution time model, failure intensity is given by:

(a) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0^2}\right)$ · (b) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$ · (c) $\lambda(\mu) = \lambda_0 \left(1 - \frac{v_0}{\mu}\right)$ · (d) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu^2}{v_0}\right)$

(b) $\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$

In Musa's Basic Execution Time Model, the failure intensity $\lambda(\mu)$ decreases linearly as a function of the cumulative failures experienced (μ):

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

Where:

- λ_0**: Initial failure intensity at the beginning of execution ($\mu = 0$).**
- v_0**: Total number of failures that would be experienced in infinite execution time.**
- μ**: Cumulative number of failures experienced up to that point.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 330; companion slides 48, 112.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة شدة الفشل في النموذج الأساسي دالة خطية تنازلية: $\lambda(\mu) = \lambda_0(1 - \mu/v_0)$. تنخفض الشدة خطياً مع ازدياد الأعطال المكتشفة والمصلحة.

Q7.21. In the Basic execution time model, the additional number of failures required to achieve a failure intensity objective is expressed as:

(a) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_F - \lambda_P)$ · (b) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$ · (c) $\Delta\mu = \frac{\lambda_0}{v_0}(\lambda_P - \lambda_F)$ · (d) $\Delta\mu = \frac{\lambda_0}{v_0}(\lambda_F - \lambda_P)$

(b) $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$

Rearranging the linear failure intensity relationship for the present intensity (λ_P) and objective intensity (λ_F):

$$\lambda_P = \lambda_0\left(1 - \frac{\mu_P}{v_0}\right), \quad \lambda_F = \lambda_0\left(1 - \frac{\mu_F}{v_0}\right)$$

Subtracting the two equations yields:

$$\lambda_P - \lambda_F = \frac{\lambda_0}{v_0}(\mu_F - \mu_P) = \frac{\lambda_0}{v_0}\Delta\mu$$

Solving for $\Delta\mu$ gives:

$$\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 331; companion slides 51, 112.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- عدد الأعطال الإضافية اللازمة للوصول إلى هدف الاعتمادية هو: $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$. انتبه لطرح الهدف المنخفض من الشدة الحالية $(\lambda_P - \lambda_F)$.

Q7.22. In the Basic execution time model, the additional execution time required to achieve a failure intensity objective is given as:

(a) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_F}{\lambda_P}\right)$ · (b) $\Delta\tau = \frac{\lambda_0}{v_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$ · (c) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$ · (d) $\Delta\tau = \frac{\lambda_0}{v_0} \ln\left(\frac{\lambda_F}{\lambda_P}\right)$

(c) $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$

In the Basic execution time model, failure intensity decays exponentially with execution time τ:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

Evaluating between current intensity λ_P and target intensity λ_F:

$$\frac{\lambda_P}{\lambda_F} = \exp\left(\frac{\lambda_0}{v_0}\Delta\tau\right)$$

Taking the natural logarithm of both sides and solving for $\Delta\tau$:

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 331; companion slides 52, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- زمن التنفيذ الإضافي المطلوب للوصول لهدف الاختبار: $\Delta\tau = \frac{v_0}{\lambda_0}\ln(\lambda_P/\lambda_F)$. النسبة داخل اللوغاريتم هي القيمة الأكبر على الأصغر (λ_P/λ_F) لضمان ناتج موجب.

Q7.23. The failure intensity function of the Logarithmic Poisson execution time model is given as:

(a) $\lambda(\mu) = \lambda_0 \ln(-\theta\mu)$ · (b) $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$ · (c) $\lambda(\mu) = \lambda_0 \exp(\theta\mu)$ · (d) $\lambda(\mu) = \lambda_0 \log(\theta\mu)$

(b) $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$

In the Logarithmic Poisson execution time model (Musa–Okumoto, 1984), failure intensity decreases exponentially with respect to cumulative failures experienced (μ):

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

Where θ is the failure intensity decay parameter. This exponential decay captures the realistic observation that early faults contribute much more heavily to failure intensity than subtle faults discovered late in testing.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 335; companion slides 59, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة شدة الفشل في نموذج بواسون اللوغاريتمي تنخفض أسياً مع عدد الأخطاء: $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$.

Q7.24. In the Logarithmic Poisson execution time model, "θ" is known as:

(a) Failure intensity function parameter · (b) Failure intensity decay parameter · (c) Failure intensity measurement · (d) Failure intensity increment parameter

(b) Failure intensity decay parameter

In the Logarithmic Poisson formulation, θ is explicitly termed the failure intensity decay parameter (measured in failure^{-1}). It quantifies the relative reduction in failure intensity per failure experienced:

$$\theta = -\frac{d\lambda/d\mu}{\lambda}$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 335; companion slides 59, 113.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الرمز θ في نموذج بواسون اللوغاريتمي يمثل معامل تلاشي شدة الفشل (Failure intensity decay parameter).

Q7.25. In the Jelinski–Moranda model, failure intensity is defined as:

(a) $\lambda(t_i) = \phi[N - (i-1)]$ · (b) $\lambda(t_i) = \phi[N + (i-1)]$ · (c) $\lambda(t_i) = \phi[N - i]$ · (d) $\lambda(t_i) = \phi[N/i]$

(a) $\lambda(t_i) = \phi[N - (i-1)]$

The Jelinski–Moranda model (1972) assumes that software contains an initial total of N faults, each contributing an equal hazard rate ϕ. When the i-th failure occurs, $(i-1)$ faults have already been completely and perfectly removed without introducing new faults. Thus, during the interval between the $(i-1)$-th and i-th failure, the failure intensity is:

$$\lambda(t_i) = \phi[N - (i-1)]$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 345; companion slides 90, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معادلة جيلينسكي-موراندا: شدة الفشل عند العطل رقم i تتناسب مع عدد الأخطاء المتبقية: $\phi[N - (i-1)]$.

Q7.26. CMM Level 1 has:

(a) 6 KPAs · (b) 2 KPAs · (c) 0 KPAs · (d) None of the above

(c) 0 KPAs

In the Capability Maturity Model (CMM), Level 1 (Initial) has exactly zero Key Process Areas (KPAs). At Level 1, processes are ad-hoc, chaotic, and undefined; project success depends solely on individual heroism and crisis management rather than repeatable institutional practices. KPAs appear beginning at Level 2 (which has 6 KPAs).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المستوى الأول في نموذج CMM (المستوى الابتدائي Initial) يحتوي على **صفر من مناطق العمليات الرئيسية (0 KPAs)** لأن العمل فيه فوضوي وغير خاضع لمعايير مؤسسية.

Q7.27. MTBF stands for:

(a) Mean time between failure · (b) Maximum time between failures · (c) Minimum time between failures · (d) Many time between failures

(a) Mean time between failure

MTBF stands for Mean Time Between Failures. It represents the average elapsed operational time separating successive system failures in repairable systems, calculated as the sum of Mean Time To Failure (MTTF) and Mean Time To Repair (MTTR):

$$MTBF = MTTF + MTTR$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 13, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح MTBF يعني متوسط الزمن بين الانهيارات (Mean Time Between Failures) في الأنظمة القابلة للإصلاح.

Q7.28. CMM model is a technique to:

(a) Improve the software process · (b) Automatically develop the software · (c) Test the software · (d) All of the above

(a) Improve the software process

The Capability Maturity Model (SEI CMM) is an organizational framework designed specifically to assess the maturity of software processes and guide organizations through evolutionary stages of software process improvement. It does not automatically generate source code or function as an automated testing engine.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- هدف CMM الأساسي هو تحسين نضج العملية البرمجية للمؤسسة (Improve the software process)، وليس توليد كود آلياً.

Q7.29. Total number of maturity levels in CMM are:

(a) 1 · (b) 3 · (c) 5 · (d) 7

(c) 5

The SEI Capability Maturity Model defines exactly five evolutionary maturity levels:

1. **Level 1 — Initial:** Chaotic, ad-hoc performance.
2. **Level 2 — Repeatable:** Basic project management controls.
3. **Level 3 — Defined:** Standardized, institutionalized engineering processes.
4. **Level 4 — Managed:** Quantitative process measurement and product quality metrics.
5. **Level 5 — Optimizing:** Continuous process improvement through feedback and technological innovation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 114.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مستويات CMM الخمسة بالترتيب: 1- Initial ← 2- Repeatable ← 3- Defined ← 4- Managed ← 5- Optimizing.

Q7.30. Reliability of a software is dependent on number of errors:

(a) Removed · (b) Remaining · (c) Both (a) & (b) · (d) None of the above

(b) Remaining

Software reliability is determined strictly by the latent faults currently remaining in the code that can be encountered during execution, combined with the frequency of their activation under the operational profile. The historical count of faults removed during past debugging sessions does not dictate current system stability; a system from which 1,000 faults were removed may still fail immediately if critical residual faults remain in frequently executed pathways.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 309, 313; companion slides 9, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فخ امتحاني شهير: اعتمادية النظام تعتمد حصراً على عدد الأخطاء المتبقية (Remaining) في الكود أثناء التشغيل، ولا تعتمد على عدد الأخطاء التي تم إصلاحها في الماضي.

Q7.31. Reliability of software is usually estimated at:

(a) Analysis phase · (b) Design phase · (c) Coding phase · (d) Testing phase

(d) Testing phase

Dynamic software reliability models (such as Musa's models and Jelinski–Moranda) require executing the software under an operational profile and collecting dynamic failure timestamps and counts. Consequently, statistical estimation and measurement of software reliability take place primarily during system and integration **testing**, once an executable system exists.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 312–313; companion slides 10, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تقدير الاعتمادية إحصائياً يحتاج إلى تشغيل الكود وتسجيل أزمنة الفشل، ولذلك يتم قياسها في مرحلة الاختبار (Testing phase).

Q7.32. CMM stands for:

(a) Capacity maturity model · (b) Capability maturity model · (c) Cost management model · (d) Comprehensive maintenance model

(b) Capability maturity model

CMM stands for Capability Maturity Model, developed by the Software Engineering Institute (SEI) at Carnegie Mellon University.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- CMM اختصار لـ Capability Maturity Model.

Q7.33. Which level of CMM is for basic project management?

(a) Initial · (b) Repeatable · (c) Defined · (d) Managed

(b) Repeatable

Level 2 (Repeatable) of the Capability Maturity Model establishes fundamental project management controls, focusing on project planning, requirements management, project tracking, subcontract management, quality assurance, and configuration management to ensure realistic commitments based on past project successes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 348; companion slides 95, 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- المستوى الثاني (Repeatable) هو المستوى المسؤول عن إدارة المشاريع الأساسية (Basic project management).

Q7.34. Which level of CMM is for process control?

(a) Initial · (b) Repeatable · (c) Defined · (d) Managed

(d) Managed

Level 4 (Managed) uses quantitative measures to control process and product quality.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 361; companion slide 115.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- ضبط والتحكم في العمليات (Process control) كمياً هو ميزة المستوى الرابع (Managed).
- الشريحة 115 تكرر صياغة سؤال 7.35؛ صيغة هذا السؤال عن Process control.

Q7.35. Which level of CMM is for process management?

(a) Initial · (b) Defined · (c) Managed · (d) Optimizing

(b) Defined

At Level 3 (Defined), standard software engineering and management processes are documented, standardized, and integrated into an organization-wide standard software process (institutional process management). Projects tailor these standard organizational processes to develop approved, managed project processes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 348–349, 361; companion slide 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- توثيق وإدارة العمليات المؤسسية القياسية (Process management) هو عنوان المستوى الثالث (Defined).

Q7.36. CMM was developed at:

(a) Harvard University · (b) Cambridge University · (c) Carnegie Mellon University · (d) Maryland University

(c) Carnegie Mellon University

The Capability Maturity Model (CMM) was developed by the Software Engineering Institute (SEI), a federally funded research and development center established at Carnegie Mellon University (CMU) in Pittsburgh, Pennsylvania.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- طور معهد SEI نموذج CMM في جامعة كارنيغي ميلون (Carnegie Mellon University).

Q7.37. McCall has developed a:

(a) Quality model · (b) Process model · (c) Reliability model · (d) None of the above

(a) Quality model

Jim McCall and his associates developed a landmark Software Quality Model in 1977 for the US Air Force, pioneering the decomposition of software quality into factors, criteria, and metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- وضع ماكول (McCall) نموذجاً لجودة البرمجيات (Quality model).

Q7.38. The model to measure software process improvement is called:

(a) ISO 9000 · (b) ISO 9126 · (c) CMM · (d) Spiral model

(c) CMM

The Capability Maturity Model (CMM) provides a structured metric and staging framework specifically designed to evaluate organizational process capability and track continuous process improvement. ISO 9126 measures product quality, ISO 9000 is a baseline certification standard, and the Spiral model is a life-cycle methodology.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 347; companion slides 94, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الموديل المخصص لقياس تحسين نضج العملية البرمجية وتدرجها هو CMM.

Q7.39. The number of clauses used in ISO 9001 are:

(a) 15 · (b) 25 · (c) 20 · (d) 10

(c) 20

ISO 9001:1994 organized its quality-system requirements into 20 clauses, numbered 4.1 through 4.20.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 355; companion slides 103, 116.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- اعتمدت مواصفة ISO 9001 (إصدار 1994 المنهجي في الكتاب) على **20 بنداً أو فقرة متطلبات (clauses 20)** تم ربطها مع متطلبات CMM.

Q7.40. ISO 9126 contains definitions of:

(a) Quality characteristics · (b) Quality factors · (c) Quality attributes · (d) All of the above

(a) Quality characteristics

The ISO/IEC 9126 standard formally structures software product quality into six top-level quality characteristics: Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability. Each characteristic is further refined into subcharacteristics and measurable metrics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 326–327; companion slides 41, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- في معيار ISO 9126، يسمى المستوى الأعلى بخصائص الجودة (Quality characteristics)، وعددها 6 خصائص رئيسية.

Q7.41. In ISO 9126, each characteristic is related to:

(a) One attribute · (b) Two attributes · (c) Three attributes · (d) Four attributes

No single option is correct for all six characteristics. The listed ISO 9126 table gives different numbers of subcharacteristics: Reliability has three, Efficiency has two, and Functionality has four.

The MCQ is therefore inconsistent as worded.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 327; companion slides 41, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يرتبط كل بعد رئيسي في سلايدات أغروال بصفات فرعية محددة (Subcharacteristics) تمثل السمات المشتقة.
- الشريحة 117 تضع الخيار (c)، لكن جدول الكتاب يبيّن أن العدد يختلف حسب الخاصية؛ لذلك لا يوجد خيار واحد يصح للجميع.

Q7.42. In McCall quality model, product revision quality factor consists of:

(a) Maintainability, Flexibility, Testability · (b) Portability, Reusability, Interoperability · (c) Correctness, Reliability, Efficiency · (d) Usability, Integrity, Efficiency

(a) Maintainability, Flexibility, Testability

Under McCall's framework, Product Revision focuses on how easily the software can be inspected, repaired, and enhanced. It consists of exactly three factors:

1. **Maintainability:** Effort required to locate and fix defects.
2. **Flexibility:** Effort required to modify the software for new operational needs.
3. **Testability:** Effort required to validate that the software performs its intended functions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 33, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- عوامل مراجعة المنتج (Product Revision) في نموذج ماكول هي ثلاثية الصيانة: سهولة الصيانة (Maintainability) + المرونة (Flexibility) + القابلية للاختبار (Testability).

Q7.43. Which is not a software reliability model?

(a) The Jelinski–Moranda Model · (b) Basic execution time model · (c) Spiral model · (d) None of the above

(c) Spiral model

The Spiral model is an iterative, risk-driven Software Development Life Cycle (SDLC) process model formulated by Barry Boehm. It is not a statistical software reliability growth model. Jelinski–Moranda and Musa's Basic execution time model are established reliability models.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 329, 345; companion slides 47, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- النموذج الحلزوني (Spiral model) هو نموذج دورة حياة لتطوير البرمجيات (SDLC) يعتمد على إدارة المخاطر، وليس نموذجاً رياضياً لقياس الاعتمادية.

Q7.44. Each maturity level in CMM has:

(a) One KPA · (b) Equal KPAs · (c) Several KPAs · (d) No KPA

(c) Several KPAs

With the exception of Level 1 (which has 0 KPAs), each mature level in the CMM comprises multiple Key Process Areas (KPAs): Level 2 has 6 KPAs, Level 3 has 7 KPAs, Level 4 has 2 KPAs, and Level 5 has 3 KPAs.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 350–352; companion slides 97–101, 117.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- يضم كل مستوى من مستويات CMM الناضجة عدة مناطق عمليات رئيسية (Several KPAs)، تتراوح بين 2 و7 مجالات.

Q7.45. KPA in CMM stands for:

(a) Key Process Area · (b) Key Product Area · (c) Key Principal Area · (d) Key Performance Area

(a) Key Process Area

KPA stands for Key Process Area. In CMM, a Key Process Area identifies a cluster of related activities that, when performed collectively, achieve a set of goals considered critical for establishing process capability at that maturity level.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 350; companion slides 97, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مصطلح KPA يعني منطقة العملية الرئيسية (Key Process Area).

Q7.46. In reliability models, our emphasis is on:

(a) Errors · (b) Faults · (c) Failures · (d) Bugs

(c) Failures

Dynamic software reliability models model the observable behavior of the software from the user's perspective; hence, the primary modeling emphasis is on failures (dynamic departure from required operation). Users experience failures, not passive latent faults residing silently in unexecuted source code.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 310, 329; companion slides 8, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تركز نماذج الاعتمادية على حالات الفشل (Failures) لأنها الحدث الديناميكي الحقيقي الذي يواجهه المستخدم ويؤثر على الخدمة في الواقع.

Q7.47. Software does not break or wear out like hardware. What is your opinion?

(a) True · (b) False · (c) Partially True · (d) Cannot say

(a) True

True in the physical sense: software does not wear out through physical aging. It may become obsolete or fail when its environment or requirements change, but those changes are different from hardware wear-out.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 308; companion slides 4, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- العبارة صحيحة تماماً (True): البرمجيات لا تهترئ ولا تبلى مادياً (Does not wear out) لأنها كيان منطقي غير خاضع لقوانين الاحتكاك والفيزياء.

Q7.48. Software reliability is defined with respect to:

(a) Time · (b) Speed · (c) Quality · (d) None of the above

(a) Time

Software reliability is mathematically defined as a time-dependent probability distribution $R(t)$ measuring the likelihood of failure-free operation over a specified operational time interval. Without specifying a temporal baseline (execution time, clock time, or calendar time), reliability has no mathematical meaning.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 309; companion slides 6, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- دالة الاعتمادية تُعرّف رياضياً بالنسبة إلى الزمن (Time).

Q7.49. MTTF stands for:

(a) Mean time to failure · (b) Maximum time to failure · (c) Minimum time to failure · (d) None of the above

(a) Mean time to failure

MTTF stands for Mean Time To Failure. It is the expected statistical value of the operational execution time of a software system before its next failure event occurs, defined as the integral of the reliability function:

$$MTTF = \int_0^\infty R(t)\,dt$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 312; companion slides 13, 118.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- اختصار MTTF يعني متوسط زمن حدوث الفشل (Mean Time To Failure).

Q7.50. ISO 9000 is a series of standards for quality management systems and has:

(a) 2 related standards · (b) 5 related standards · (c) 10 related standards · (d) 25 related standards

(b) 5 related standards

The original ISO 9000 series comprises five related standards:

1. **ISO 9000:** Quality management and quality assurance standards — Guidelines for selection and use.
2. **ISO 9001:** Model for quality assurance in design, development, production, installation, and servicing.
3. **ISO 9002:** Model for quality assurance in production, installation, and servicing.
4. **ISO 9003:** Model for quality assurance in final inspection and test.
5. **ISO 9004:** Quality management and quality system elements — Guidelines.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 354; companion slides 103, 119.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- تتكون سلسلة معايير ISO 9000 الأصلية من **5 مواصفات مرتبطة** شهيرة (ISO 9000, 9001, 9002, 9003, 9004).

Section 2: Syllabus Gap-Closing Synthesis MCQs (10 Items)

Q7.51. In Harlan Mills' Bug Seeding Model, if S artificial faults are seeded into a program, and testing subsequently uncovers s seeded faults and n original unseeded faults, the estimated number of latent unseeded faults $\hat{N}$ is:

(a) $\hat{N} = \frac{S \cdot n}{s}$ · (b) $\hat{N} = \frac{s \cdot n}{S}$ · (c) $\hat{N} = \frac{S \cdot s}{n}$ · (d) $\hat{N} = S \cdot n \cdot s$

(a) $\hat{N} = \frac{S \cdot n}{s}$

The Bug Seeding (or Tag-Recapture) model developed by Harlan Mills assumes that seeded artificial faults and intrinsic unseeded faults have equal probability of discovery. Therefore, the ratio of caught seeded faults to total seeded faults equals the ratio of caught unseeded faults to total unseeded faults:

$$\frac{s}{S} = \frac{n}{\hat{N}} \implies \hat{N} = \frac{S \cdot n}{s}$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 346–347; companion slide 93.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- معادلة بذر الأخطاء (Bug Seeding) لميلز تعتمد على التناسب المتساوي لصيد الأخطاء: $\hat{N} = (S \times n)/s$.

Q7.52. In software reliability engineering, what is an "Operational Profile"?

(a) The compiler optimization configuration · (b) A quantitative set of operational run types and their associated probabilities of occurrence · (c) The hardware specification of the deployment server · (d) The developer's daily work schedule

(b) A quantitative set of operational run types and their associated probabilities of occurrence

An Operational Profile is a complete set of operations that a software system performs, together with their individual probabilities of occurrence during actual field usage. Testing according to the operational profile gives more testing effort to frequently executed operations.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 318–320; companion slides 16–20.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- ملف التشغيل (Operational Profile) يحدد أنواع العمليات في النظام مع احتمالية تشغيل كل منها في الواقع، لتوجيه الاختبار نحو المسارات الأكثر استخداماً.

Q7.53. In Musa's Calendar Time Component modeling, during which operational phase does testing progress slow down primarily due to personnel limitations rather than compute constraints?

(a) Test-limiting phase · (b) Failure-identification personnel limited phase · (c) Data-entry limiting phase · (d) Overclocked phase

(b) Failure-identification personnel limited phase

Musa's Calendar Time Component models testing progress under three resource constraints:

1. **Failure identification personnel:** Bounded by available system test engineers.
2. **Failure resolution personnel:** Bounded by development debugging staff.
3. **Computer execution resources:** Bounded by available CPU test-bench time.

Early in testing when failure intensity is very high, progress is constrained strictly by the finite hours that test and debugging personnel can devote to analyzing crashes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 339–344; companion slides 72–77.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مكون الزمن التقويمي (Calendar time) يقسم الموارد إلى: موظفي تشخيص الفشل، وموظفي الإصلاح، وزمن المعالج. في بداية الاختبار عندما تتساقط الأخطاء بكثرة، يكون المورد الحاكم المقيد هو كادر التحقيق والإصلاح.

Q7.54. In the Capability Maturity Model (CMM), what are the "Common Features" that characterize every Key Process Area?

(a) Input, Process, Output, Storage · (b) Commitment to Perform, Ability to Perform, Activities Performed, Measurement & Analysis, Verifying Implementation · (c) Goals, Metrics, Actions, Feedback · (d) Planning, Execution, Review, Retirement

(b) Commitment to Perform, Ability to Perform, Activities Performed, Measurement & Analysis, Verifying Implementation

In SEI CMM, each KPA is organized across five structural Common Features:

1. **Commitment to Perform:** Organizational policies and management leadership.
2. **Ability to Perform:** Resources, funding, tools, and training preconditions.
3. **Activities Performed:** Specific engineering and management tasks.
4. **Measurement and Analysis:** Basic metrics used to track process performance.
5. **Verifying Implementation:** Objective reviews and audits by SQA and executive management.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 353–354; companion slide 102.

📝 **Academic Note**

ملاحظة الطالب والفخ الامتحاني:

- السمات المشتركة (Common Features) الخمس التي تبني أي KPA في CMM هي: الالتزام بالأداء ← القدرة على الأداء ← الأنشطة المنفذة ← القياس والتحليل ← التحقق من التنفيذ.

Q7.55. What is the main difference in emphasis between ISO 9001 and CMM?

(a) ISO 9001 emphasizes continuous software-process improvement; CMM defines only minimum quality-system criteria · (b) CMM emphasizes continuous process improvement, while ISO 9001 defines minimum criteria for an adequate quality-management system · (c) ISO 9001 applies only to software; CMM applies to all industries · (d) CMM concerns documentation only, while ISO 9001 provides a staged software-improvement path

(b) CMM emphasizes continuous process improvement, while ISO 9001 defines minimum criteria for an adequate quality-management system

CMM places greater emphasis on continuous process improvement. ISO 9001 defines minimum criteria for an adequate quality-management system.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.5.2–§7.5.3, pp. 356–357; companion slide 107.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الفرق المحوري هو أن CMM يركز أكثر على التحسين المستمر للعمليات، بينما يحدد ISO 9001 الحد الأدنى لنظام إدارة جودة مقبول.

Q7.56. In McCall's Software Quality Model, which factor belongs to the "Product Transition" dimension?

(a) Correctness · (b) Flexibility · (c) Portability · (d) Usability

(c) Portability

Product Transition encompasses three factors concerned with moving the software to new operating environments: Portability, Reusability, and Interoperability. Correctness and Usability belong to Product Operation; Flexibility belongs to Product Revision.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 323; companion slides 30–35.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- عوامل الانتقال (Product Transition) الثلاثة في ماكول هي: القابلية للنقل (Portability) وإعادة الاستخدام (Reusability) والتشغيل البيني (Interoperability).

Q7.57. In the Logarithmic Poisson execution time model, how does total failures experienced behave as execution time $\tau \to \infty$?

(a) It reaches a finite upper limit v_0 · (b) It grows logarithmically without an infinite upper bound ($\mu \to \infty$) · (c) It drops immediately to zero · (d) It becomes negative

(b) It grows logarithmically without an infinite upper bound ($\mu \to \infty$)

A distinguishing mathematical attribute of the Musa–Okumoto Logarithmic Poisson model is that it models an infinite-failure category:

$$\mu(\tau) = \frac{1}{\theta} \ln(\lambda_0 \theta \tau + 1)$$

As $\tau \to \infty$, $\mu(\tau) \to \infty$ logarithmically. Unlike the Basic model (which posits a finite total fault count v_0), the Logarithmic Poisson model assumes that software will continue to generate failures indefinitely at increasingly tiny failure intensities as software complexity and operational duration grow unbounded.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 335–336; companion slides 59–60.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- فرق جوهري بين النموذجين: النموذج الأساسي يفترض سقفاً نهائياً لعدد الأعطال (v_0)، بينما نموذج بواسون اللوغاريتمي يفترض أن الأعطال المتوقعة تتزايد لوغاريتمياً دون سقف أعلى محدود مع الزمن اللانهائي.

Q7.58. Which Key Process Area (KPA) belongs to CMM Level 2 (Repeatable)?

(a) Organization Process Definition · (b) Software Configuration Management (SCM) · (c) Quantitative Process Management · (d) Defect Prevention

(b) Software Configuration Management (SCM)

Level 2 (Repeatable) comprises exactly six KPAs focused on project-level tracking and discipline:

1. Requirements Management
2. Software Project Planning
3. Software Project Tracking and Oversight
4. Software Subcontract Management
5. Software Quality Assurance
6. **Software Configuration Management**

Organization Process Definition is Level 3; Quantitative Process Management is Level 4; Defect Prevention is Level 5.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), p. 350; companion slides 97, 101.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- مناطق العمليات الست للمستوى الثاني (Level 2): إدارة المتطلبات، تخطيط المشاريع، التتبع والرقابة، إدارة التعاقدات الفرعية، ضمان جودة البرمجيات (SQA)، وإدارة التكوين (SCM).

Q7.59. The relationship between execution time τ and calendar time t in software reliability is mediated by:

(a) The compiler execution clock · (b) Resource allocation parameters representing available personnel, compute time, and their utilization ratios · (c) The lines of source code · (d) Function point complexity adjustment

(b) Resource allocation parameters representing available personnel, compute time, and their utilization ratios

Execution time measures CPU usage; calendar time also depends on the resources available and how fully each resource can be utilized. The limiting resource determines the calendar-time to execution-time ratio:

$$\frac{dt}{d\tau} = \max_{r \in \{I,F,C\}} \left(\frac{1}{P_r p_r} \frac{d\chi_r}{d\tau} \right) = \max_{r \in \{I,F,C\}} \left(\frac{\theta_r + \mu_r \lambda}{P_r p_r} \right)$$

Here, P_r is the available quantity of resource r, p_r is its utilization fraction, and χ_r is its cumulative usage. The terms θ_r and μ_r are resource usage per CPU hour and per failure, respectively; λ is failure intensity.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 339–344; companion slides 72–77.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- التحويل من زمن التنفيذ (τ) إلى الزمن التقويمي (t) يتحكم به توفر الموارد البشرية (كادر الفحص وكادر الإصلاح) وقدرات أجهزة المعالجة.

Q7.60. In CMM process maturity staging, which level introduces statistical process control (SPC) and quantitative process capability baselines?

(a) Level 2 · (b) Level 3 · (c) Level 4 · (d) Level 5

(c) Level 4

Level 4 (Managed) introduces quantitative management and statistical process control (SPC). Productivity and quality metrics are measured across all process activities, establishing statistical process capability baselines that allow management to distinguish common cause variation from special cause variation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), pp. 348–349; companion slides 95, 100.

Academic Note

ملاحظة الطالب والفخ الامتحاني:

- الضبط الإحصائي للعمليات (Statistical Process Control) والقياس الكمي الدقيق يبدأ حصراً في المستوى الرابع (Level 4: Managed).

ASE CHAPTER 7

T2 Descriptive & Short-Answer Pack

Question 7.1 · Software Reliability Definition & Operational Existence (Exercise 7.1)

What is software reliability? Does it exist?

Software reliability is defined as the probability of failure-free software operation for a specified period of time in a specified environment. It is formally expressed as:

$$R(t) = P(\text{No failures occur in the interval } [0, t] \mid \text{specified operational environment})$$

Does it exist? Yes, software reliability genuinely exists as an operational, measurable, and probabilistic property of software execution. Although software is a logical entity that does not suffer from physical wear-and-tear, corrosion, or material fatigue like hardware, software reliability exists because: - **Input sequence uncertainty:** Operational execution selects a dynamic sequence of inputs from an extremely large input space. - **Latent design faults:** Software contains latent specification, design, and coding faults introduced during development. - **Failure manifestation:** When the operational trajectory encounters an input state that activates an uncorrected fault, an observable failure occurs. Software reliability quantifies the probability that the system executes without encountering such fault-triggering input states over a designated operating interval.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة امتحانية: البرمجيات لا تبلى فيزيائياً مثل العتاد، لكن موثوقيتها حقيقية وتُقاس باحتمالية تنفيذ المدخلات من دون تفعيل عيوب برمجية كامنة خلال زمن تشغيل محدد ← التعريف الرسمي المعتمد يربط الموثوقية بالاحتمالية، والزمن، وبيئة التشغيل المحددة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1 & §7.1.1, pp. 308–310; companion slides 6–8.

Question 7.2 · Software Failure vs Software Fault (Exercise 7.4)

What is software failure? How is it related with a fault?

A software failure is an observable departure of the external system behavior or output from user requirements or specified system behavior during execution. It is a dynamic runtime event that occurs only when the software is actively executed.

A software fault (commonly called a defect or bug) is an internal structural flaw, incorrect step, erroneous process, or incorrect data definition in a software program. It is a static entity that resides within the source code, design specifications, or configuration data.

Relationship between Fault and Failure: - **Causal relationship: A fault is the latent internal cause; a failure is the dynamic external manifestation.** - **Activation requirement:** A fault remains dormant until an input state from the execution environment triggers the faulty code path and propagates incorrect

internal states to the observable system boundary. - **Non-equivalence:** Not every execution of faulty code produces an immediate failure (e.g., if erroneous data is masked or overwritten before reaching the output). Conversely, a single design fault may cause multiple failures across different test runs, or multiple interacting faults may cause a single system crash.

ملاحظة أكاديمية هامة (Academic Note)

الفخ الامتحاني: العيب (Fault) كيان استاتيكي ساكن في الكود ← والفشل (Failure) حدث ديناميكي مرصود أثناء التشغيل ← لا يشترط أن يؤدي كل عيب إلى فشل فوري، لكن كل فشل ينتج عن عيب كامن تم تفعيله بمدخلات معينة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.2, pp. 310–312; companion slides 8–10.

Question 7.3 · Characterizing Failure Occurrences with Respect to Time (Exercise 7.5)

Discuss the various ways of characterizing failure occurrences with respect to time.

In software reliability modeling, failure occurrences over time are characterized through four distinct mathematical and observational representations:

1. **Time of Failure:**
2. **Recording the exact elapsed operating times $t_1, t_2, t_3, \ldots, t_i$ at which successive failures occur**, measured from the beginning of execution testing ($t = 0$).
3. **Time Interval Between Failures:**
4. **The elapsed operating time T_i between two successive failure occurrences**, defined as:

$$T_i = t_i - t_{i-1}$$

5. This represents failure inter-arrival times and is modeled as a random variable.
6. **Cumulative Failures in a Given Time Interval:**
7. **The total number of failures experienced from time zero up to a specified execution time t**, denoted as $\mu(t)$ (the mean value function) or counting discrete events $M(t)$.
8. **Failure Intensity:**
9. **The instantaneous rate of failure occurrences per unit of execution time**, defined as the derivative of the expected cumulative failure function:

$$\lambda(t) = \frac{d\mu(t)}{dt}$$

10. Typically expressed in units of failures per CPU hour or failures per calendar month.

ملاحظة أكاديمية هامة (Academic Note)

تنبيه: هناك 4 طرق رئيسية لتمثيل حدوث الفشل عبر الزمن: زمن حدوث الفشل الفردي (t_i)، الفترة الفاصلة بين فشلين (T_i)، العدد التراكمي للفشل ($\mu(t)$)، وكثافة الفشل الفورية ($\lambda(t)$) ← السؤال يطلب هذه الأربعة حصراً.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.3, pp. 312–314; companion slides 11–15.

Question 7.4 · Core Reliability Terminology (Exercise 7.6)

Describe the following terms: 1. Operational Profile 2. Input Space 3. MTBF (Mean Time Between Failures) 4. MTTF (Mean Time To Failure) 5. Failure Intensity

1. **Operational Profile:**
2. **A quantitative specification of the complete set of operational tasks or transactions executed by a software system, paired with their respective probabilities of occurrence in actual customer use.**
3. It guides realistic, usage-based reliability testing by ensuring the most frequently executed user scenarios receive proportional test coverage.
4. **Input Space:**
5. **The complete mathematical set of all possible input vectors, data states, and environmental conditions that a software system can receive and process during execution.**
6. Only a tiny fraction of the input space activates latent faults (termed the *failure domain*).
7. **MTBF (Mean Time Between Failures):**
8. **The expected operating time between consecutive failures for a repairable system.**
9. It incorporates both operational operating time and the time required to diagnose, repair, and restart the software:

$$\mathrm{MTBF} = \mathrm{MTTF} + \mathrm{MTTR}$$

10. **MTTF (Mean Time To Failure):**
11. **The expected average operating time of a system from the start of operation until the occurrence of the next failure.**
12. For a constant failure rate λ, it is given by $\mathrm{MTTF} = \frac{1}{\lambda}$.
13. **Failure Intensity (λ):**
14. **The number of software failures occurring per unit of execution time**, representing the instantaneous failure frequency of the system (expressed as failures per CPU hour).

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة: العلاقة الحاكمة بين مؤشرات الزمن: $\mathrm{MTBF} = \mathrm{MTTF} + \mathrm{MTTR}$ ← للأنظمة البرمجية القابلة للإصلاح، الفرق هو زمن الاستعادة والإصلاح (MTTR).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.4–§7.1.7, pp. 314–318; companion slides 16–23.

Question 7.5 · Uses of Reliability Studies in Software Engineering (Exercise 7.7)

What are the uses of reliability studies? How can one use software reliability measures to monitor the operational performance of software?

Reliability studies serve multiple core engineering and management functions across the software lifecycle:

- **Release Readiness Certification:** Providing quantitative, objective criteria to decide when testing can terminate and software is ready for commercial deployment (e.g., reaching a target failure intensity $\lambda \leq \lambda_F$).
- **Testing Progress & Trajectory Tracking:** Evaluating whether reliability is actively growing as faults are repaired, or if testing has plateaued due to inadequate test case diversity or regression defects.
- **Resource and Schedule Estimation:** Projecting the additional execution time, calendar days, and debugging effort required to meet customer quality commitments.
- **Maintenance Planning:** Forecasting field failure rates to estimate customer support staffing, warranty liabilities, and patching schedules.

Monitoring Operational Performance:

- **Operational failure logging:** Measuring the mean time between failures (MTBF) and failure intensity during live production across different customer operational profiles.
- **Threshold alerting:** Comparing observed operational failure rates against service level agreements (SLAs) and baseline reliability models to detect degradation caused by environment changes, hardware shifts, or faulty software updates.

ملاحظة أكاديمية هامة (Academic Note)

الفائدة الهندسية: دراسات الموثوقية تحول قرار إطلاق البرنامج (Release Decision) من مجرد تخمين زمني أو نفاد ميزانية إلى قرار كمي مبني على الوصول لكثافة الفشل المستهدفة ($\lambda \leq \lambda_F$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.8, pp. 320–322; companion slides 21–23.

Question 7.6 · Software Quality Definition & Quality Attributes (Exercise 7.8)

What is software quality? Discuss software quality attributes.

Software quality is defined as the totality of features and characteristics of a software product or service that bear on its ability to satisfy stated or implied customer needs (ISO/IEEE standard). Conformance to explicit functional requirements, documented development standards, and implicit characteristics expected of professional software.

Core Software Quality Attributes:

1. **Functionality: The degree to which the software provides functions that meet stated user requirements and specifications.**

2. **Reliability: The capability of the software to maintain its level of performance and failure-free operation under stated conditions for a specified period.**
3. **Usability: The effort and ease required for a user to learn, operate, prepare inputs for, and interpret outputs from the software.**
4. **Efficiency: The relationship between the software's performance level and the amount of computational resources (CPU time, memory, bandwidth) utilized.**
5. **Maintainability: The ease with which the software can be modified, corrected, improved, or adapted to environmental changes.**
6. **Portability: The ability of software to be transferred from one hardware/software platform or operating environment to another.**

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة: الجودة مفهوم مظلي شامل (Umbrella Concept) يضم الموثوقية كإحدى أهم سماته ← الموثوقية تجيب عن: "هل يعمل البرنامج دون فشل؟" بينما الجودة تجيب عن: "هل يلبي البرنامج كافة الاحتياجات بكفاءة وسهولة وقابلية صيانة؟".

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2, pp. 321–323; companion slides 24–29.

Question 7.7 · Software Quality Standards: Essence and Benefits (Exercise 7.9)

What do you mean by software quality standards? Illustrate their essence as well as benefits.

ISO 9001 specifies requirements for a quality system and is the standard in the ISO 9000 series most applicable to software development. The Capability Maturity Model (CMM) is a staged model for judging software-process maturity and guiding process improvement; a CMM maturity level is distinct from ISO 9001 certification.

Their practical value is to make quality-system activities documented and reviewable. ISO 9001 addresses areas such as management responsibility, document control, inspection and testing, corrective action, and quality records. CMM organizes software-process practices into maturity levels and supports continuing process improvement.

These documented activities can support consistent execution, traceable records, and process improvement.

ملاحظة أكاديمية هامة (Academic Note)

الفرق الأساسي: ISO 9001 معيار لنظام الجودة ويمكن أن يتطلب السوق شهادة له؛ أما CMM فيقيّم نضج عمليات البرمجيات ويوجه تحسينها، ومستوى النضج مو شهادة ISO.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1 & §7.4, pp. 323–325, 347–350; companion slides 27–32.

Question 7.8 · McCall Software Quality Model Structure & Product Factors (Exercise 7.10)

Describe the McCall software quality model. How many product quality factors are defined and why?

The McCall Software Quality Model (developed by Jim McCall in 1977 for the US Air Force) organizes software quality into a hierarchical framework connecting high-level management factors, intermediate technical criteria, and concrete metrics.

Structure and Factor Count: McCall defines exactly **11 product quality factors grouped under 3 operational perspectives (lifecycle phases):**

Product Operation (5 factors — operational characteristics during live execution):

- **Correctness:** The extent to which a program satisfies its specifications and user mission objectives.
- **Reliability:** The extent to which a program performs its intended function with required precision over time.
- **Efficiency:** The amount of computing resources and code required to execute a function.
- **Integrity:** The extent to which unauthorized access to software or data can be controlled.
- **Usability:** The effort required to learn, operate, prepare input, and interpret output.

Product Revision (3 factors — modifiability and maintenance characteristics):

- **Maintainability:** The effort required to locate and fix an error in an operational program.
- **Flexibility:** The effort required to modify an operational program to incorporate new requirements.
- **Testability:** The effort required to test a program to ensure that it performs its intended function.

Product Transition (3 factors — portability and external adaptability):

- **Portability:** The effort required to transfer the program from one hardware/software platform to another.
- **Reusability:** The extent to which a program or its components can be reused in other applications.
- **Interoperability:** The effort required to couple one system with another.

Why this organization? McCall structured the model around 3 perspectives to reflect the complete operational lifecycle of military and industrial software: using it daily (Operation), fixing and evolving it (Revision), and migrating or integrating it into new computing environments (Transition).

ملاحظة أكاديمية هامة (Academic Note)

قاعدة الحفظ السريع لعوامل مكول (11 عاملاً): - التشغيل (5): Correctness, Reliability, Efficiency, Integrity, Usability. - التعديل (3): Maintainability, Flexibility, Testability. - الانتقال (3): Portability, Reusability, Interoperability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.2, pp. 323–324; companion slides 30–35.

Question 7.9 · Factors vs Criteria in McCall's Model (Exercise 7.11)

Discuss the relationship between quality factors and quality criteria in McCall's software quality model.

In McCall's model, the relationship between quality factors and quality criteria represents a hierarchical decomposition from user-oriented quality desires down to developer-oriented software attributes:

- **Quality Factors (High-Level / External):** Represent external views of quality meaningful to end-users and management (e.g., *Is the software reliable?*, *Is it easy to maintain?*). Factors cannot be measured directly through a single inspection.
- **Quality Criteria (Intermediate / Internal):** Represent technical, internal software engineering attributes meaningful to software developers and architects (e.g., *Error tolerance, Simplicity, Modularity, Traceability, Consistency, Instrumentation*).

Structural Relationship: - **Multi-to-Multi Deconstructive Mapping: Each quality factor is evaluated through a composite set of related criteria.** For example, *Maintainability* is evaluated via *Simplicity, Conciseness, Self-descriptiveness, and Modularity*. - **Shared Technical Attributes:** Multiple factors often share underlying criteria. For instance, *Simplicity* contributes directly to *Maintainability, Testability, and Usability*. - **Measurement Bridge:** Software metrics measure technical criteria; the combined scores of criteria determine the assessment of the overarching quality factors.

ملاحظة أكاديمية هامة (Academic Note)

العلاقة: العامل (Factor) خارجي يراه المستخدم (مثل Maintainability) ← المعيار (Criterion) داخلي يراه المطور (مثل Simplicity و Modularity) ← القياسات (Metrics) تقيس المعايير لتحديد درجة تحقق العوامل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.2, pp. 324–325; companion slides 35–39.

Question 7.10 · ISO 9126 Quality Model: Characteristics and Sub-Characteristics (Exercise 7.13)

What is ISO 9126? What are the quality characteristics and attributes?

ISO/IEC 9126 (established in 1991) is an international standard that defines an internationally harmonized software product quality evaluation model. It defines **6 top-level quality characteristics**, decomposed into **21 sub-characteristics (attributes)**:

1. **Functionality:**
2. *Suitability:* Appropriateness of functions for specified tasks.
3. *Accuracy:* Provision of right or agreed results or effects.
4. *Interoperability:* Ability to interact with specified systems.
5. *Compliance:* Adherence to application-related standards or conventions.
6. *Security:* Prevention of unauthorized access to programs and data.
7. **Reliability:**
8. *Maturity:* Frequency of failure by faults in the software.

9. *Fault Tolerance:* Ability to maintain a specified level of performance in cases of software faults.
10. *Recoverability:* Capability to re-establish performance level and recover data following a failure.
11. **Usability:**
12. *Understandability:* Effort required to recognize logical concepts and applicability.
13. *Learnability:* Effort required for learning application usage.
14. *Operability:* Effort required for operation and operation control.
15. **Efficiency:**
16. *Time Behaviour:* Response/processing times and throughput rates.
17. *Resource Behaviour:* Amount of resources used and duration of use.
18. **Maintainability:**
19. *Analyzability:* Effort needed for diagnosis of deficiencies or failure causes.
20. *Changeability:* Effort needed for modification, fault removal, or environmental change.
21. *Stability:* Risk of unexpected effects from modifications.
22. *Testability:* Effort needed for validating the modified software.
23. **Portability:**
24. *Adaptability:* Opportunity for software adaptation to different specified environments.
25. *Installability:* Effort needed to install software in a specified environment.
26. *Conformance:* Adherence to standards or conventions relating to portability.
27. *Replaceability:* Opportunity and effort to use this software in place of other software.

ملاحظة أكاديمية هامة (Academic Note)

مفتاح الحفظ لمعيار ISO 9126: يتألف من 6 خصائص عليا: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability ← تتفرع إلى 21 سمة فرعية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.4, pp. 326–328; companion slides 41–46.

Question 7.11 · Musa Basic Execution Time Model Formulation & Derivation (Exercise 7.15)

Discuss the basic model of software reliability. How can μ and τ be calculated?

The Basic Execution Time Model (developed by John Musa) assumes that the failure intensity decreases linearly with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

where: - λ_0 is the initial failure intensity at the start of execution ($\mu = 0$). - v_0 is the total expected number of failures over infinite execution time. - μ is the cumulative number of failures experienced up to the current time.

Derivation of $\mu(\tau)$ and $\lambda(\tau)$ as Functions of Execution Time (τ): 1. Since failure intensity is the time rate of failure occurrences, $\lambda = \frac{d\mu}{d\tau}$:

$$\frac{d\mu}{d\tau} = \lambda_0 \left(1 - \frac{\mu}{v_0}\right) = \frac{\lambda_0}{v_0}(v_0 - \mu)$$

2. Separating variables and integrating from $\tau = 0$ (where $\mu = 0$) to τ:

$$\int_0^{\mu} \frac{d\mu}{v_0 - \mu} = \frac{\lambda_0}{v_0} \int_0^{\tau} d\tau$$

$$-\ln\left(\frac{v_0 - \mu}{v_0}\right) = \frac{\lambda_0}{v_0}\tau \implies \frac{v_0 - \mu}{v_0} = \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

3. Solving for failures experienced $\mu(\tau)$:

$$\mu(\tau) = v_0 \left[1 - \exp\left(-\frac{\lambda_0}{v_0}\tau\right)\right]$$

4. Substituting $\mu(\tau)$ back into $\lambda(\mu)$ yields failure intensity over time:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

- **Additional failures:** $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$.
- **Additional execution time:** $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$.

ملاحظة أكاديمية هامة (Academic Note)

خلاصة النموذج الأساسي (Basic Model): العلاقة خطية مع عدد الأخطاء المكتشفة (μ) ← ولكنها أسية تناقصية مع زمن التنفيذ (τ).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1, pp. 329–334; companion slides 47–58.

Question 7.12 · Logarithmic Poisson Execution Time Model & Increments (Exercise 7.19)

Write a short note on the Logarithmic Poisson Execution Time model. How can we calculate $\Delta\mu$ and $\Delta\tau$?

The Logarithmic Poisson Execution Time Model (developed by Musa and Okumoto) models software failure occurrences as a Non-Homogeneous Poisson Process (NHPP) where the failure intensity decreases exponentially with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

where: - λ_0 is the initial failure intensity. - θ is the **failure intensity decay parameter**, which measures the relative reduction in failure intensity per failure experienced.

Key Characteristics: - Unlike the Basic model, **the expected total number of failures over infinite time is infinite** ($v_0 = \infty$), reflecting the reality that software systems continually evolve and undergo modifications. - **Early failures cause a much larger decrease in failure intensity than later failures**,

which models non-uniform operational profiles where frequently executed code paths fail and are fixed first.

Formulas for $\mu(\tau)$ and $\lambda(\tau)$:

$$\mu(\tau) = \frac{1}{\theta}\ln(\lambda_0\theta\tau + 1)$$

$$\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}$$

- **Additional failures:** $\Delta\mu = \frac{1}{\theta}\ln\left(\frac{\lambda_P}{\lambda_F}\right)$.
- **Additional execution time:** $\Delta\tau = \frac{1}{\theta}\left(\frac{1}{\lambda_F} - \frac{1}{\lambda_P}\right)$.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة محورية بين النموذجين: - النموذج الأساسي (Basic): $\Delta\mu$ خطي، و $\Delta\tau$ لوغاريتمي. - النموذج اللوغاريتمي (Logarithmic): $\Delta\mu$ لوغاريتمي، و $\Delta\tau$ كسر معكوس مقلوب $\left(\frac{1}{\lambda_F} - \frac{1}{\lambda_P}\right)$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.2, pp. 335–339; companion slides 59–71.

Question 7.13 · Calendar Time Component Model & Time Conversion (Exercise 7.23)

Discuss the calendar time component model. Establish the relationship between calendar time to execution time.

The Calendar Time Component Model bridges the gap between software execution time (τ, measured in CPU hours) and managerially relevant calendar time (t, measured in elapsed working days or months).

Resource-Limited Segments: In any testing phase, progress is constrained by one of three limiting project resources: 1. **Computer Execution Time:** Limited by available test computers/workstations ($k = C$). 2. **Failure Identification Personnel:** Limited by available test engineers executing tests ($k = I$). 3. **Failure Correction Personnel:** Limited by debugging programmers diagnosing and fixing defects ($k = F$).

Mathematical Relationship between Calendar Time and Execution Time: Resource usage χ_k is modeled as a linear combination of execution time and failures experienced:

$$\chi_k = \theta_k\tau + \mu_k\mu$$

where θ_k is resource usage per unit execution time, and μ_k is resource usage per failure.

Differentiating with respect to execution time τ:

$$\frac{d\chi_k}{d\tau} = \theta_k + \mu_k\frac{d\mu}{d\tau} = \theta_k + \mu_k\lambda$$

The instantaneous conversion ratio from execution time to calendar time is obtained by dividing by available utilized resources $P_k\rho_k$:

$$\frac{dt}{d\tau} = \frac{1}{P_k \rho_k} \frac{d\chi_k}{d\tau} = \frac{\theta_k + \mu_k \lambda}{P_k \rho_k}$$

Integration over an interval:

$$\Delta t = \frac{\theta_k \Delta\tau + \mu_k \Delta\mu}{P_k \rho_k}$$

During testing, the governing bottleneck shifts dynamically from failure correction (high initial failure intensity) to failure identification, and finally to computer time as failures become rare.

ملاحظة أكاديمية هامة (Academic Note)

أهمية النموذج: زمن المعالج (CPU Hours) لا يفهمه مدير المشروع والزبون ← نموذج زمن التقويم يحول ساعات التنفيذ إلى أيام عمل وتاريخ تسليم حقيقي بناءً على المورد الأشد اختناقاً (Bottleneck).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.3, pp. 339–344; companion slides 72–88.

Question 7.14 · Jelinski–Moranda Reliability Model Formulation (Exercise 7.25)

Explain the Jelinski–Moranda model of reliability theory. What is the relation between t and λ?

The Jelinski–Moranda Model (1972) is one of the earliest and most fundamental software reliability growth models.

Underlying Assumptions: 1. The software contains a finite, fixed number of initial faults, denoted as N. 2. Each fault is independent and contributes equally to the overall failure hazard rate. 3. Whenever a failure occurs, the fault causing it is immediately and perfectly repaired with zero defect introduction. 4. The failure rate between successive failures remains constant.

Relation between Time (t) and Failure Rate (λ): - The hazard rate during the interval between the $(i-1)$-th and the i-th failure is a piecewise constant step function:

$$\lambda(t_i) = \phi[N - (i-1)]$$

where ϕ is a constant of proportionality representing the hazard rate per single unmanifested fault. - **Stepwise relation: The relationship between time t and hazard rate λ is a discrete step-down staircase curve.** As cumulative operational time t advances and failures occur, each repair decreases the hazard rate by a discrete step ϕ. The time interval between successive failures T_i grows stochastically larger as i approaches N.

ملاحظة أكاديمية هامة (Academic Note)

الخاصية الأساسية لنموذج Moranda–Jelinski: معدل الخطورة (Hazard rate) ثابت بين كل فشلين متتاليين، ويهبط هبوطاً درجياً (Staircase) بمقدار ϕ مع كل تصليح ناجح.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.4, pp. 343–346; companion slides 89–92.

Question 7.15 · Mills Bug Seeding Model Principles & Advantages (Exercise 7.26)

Describe the Mills bug seeding model. Discuss few advantages of this model over other reliability models.

The Mills Bug Seeding Model (introduced by Harlan Mills in 1972) is an empirical reliability estimation technique based on the classical capture-recapture ecological sampling method.

Operational Principle: 1. An independent quality officer intentionally inserts (seeds) a known number S of artificial bugs into the software codebase prior to testing. 2. The regular testing and debugging team conducts normal testing without knowledge of the count, types, or locations of the seeded bugs. 3. During testing, discovered bugs are classified into seeded bugs (s) and naturally occurring indigenous bugs (n). 4. Assuming seeded and indigenous bugs are equally distributed and equally detectable, the ratio of caught seeded bugs to total seeded bugs approximates the ratio of caught indigenous bugs to total indigenous bugs:

$$\frac{s}{S} \approx \frac{n}{N} \implies \hat{N} = \frac{S \times n}{s}$$

Advantages of Bug Seeding: - **Simple, intuitive mechanics:** Requires no complex statistical distribution fittings, differential equation solving, or continuous calendar time logging. - **Direct fault population estimation:** Estimates the absolute count of latent bugs remaining in the code ($\hat{N} - n$) directly. - **Tester efficiency metric:** The recovery ratio $\frac{s}{S}$ provides an immediate, objective measurement of test team thoroughness.

Primary Limitation: In practice, it is extremely difficult to synthesize artificial bugs that possess the exact error profile, subtlety, and execution probabilities of real human programming defects.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

معادلة ميلز الأساسية:

$$\hat{N} = \frac{S \times n}{s}$$

حيث S الأخطاء المزروعة، s المكتشف منها، n الأخطاء الأصلية المكتشفة، و $\hat{N}$ إجمالي الأخطاء الأصلية المقدرة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.5, pp. 346–347; companion slide 93.

Question 7.16 · CMM Process Improvement Philosophy (Exercise 7.27)

Explain how the CMM encourages continuous improvement of the software process.

The Capability Maturity Model (CMM) is a five-level staged approach to software-process improvement, progressing from ad hoc work at Level 1 toward continuous process improvement at Level 5.

How CMM Encourages Continuous Improvement: 1. **Progressive levels:** Level 2 establishes basic project-management controls, Level 3 defines organization-wide processes, and Level 4 measures and controls process performance quantitatively. 2. **Continuous improvement:** At Level 5, the organization

proactively identifies weaknesses, analyzes defects to determine their causes, and uses the results to prevent known defect types from recurring. 3. **Common features:** Commitment to perform, ability to perform, activities performed, measurement and analysis, and verifying implementation describe conditions used to institutionalize key process areas.

ملاحظة أكاديمية هامة (Academic Note)

تدرّج CMM: المستوى الرابع للقياس والضبط الكمي؛ المستوى الخامس للتحسين المستمر والوقاية من تكرار أنواع العيوب المعروفة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4 & §7.4.1, pp. 347–350; companion slides 94–96.

Question 7.17 · CMM Key Process Areas Across Maturity Levels (Exercise 7.28)

Discuss various key process areas of CMM at various maturity levels.

In the SEI Capability Maturity Model, **Key Process Areas (KPAs)** represent clusters of related activities that, when performed collectively, achieve a set of goals considered critical for enhancing process capability.

The 18 KPAs organized across the 5 Maturity Levels:

1. **Level 1: Initial**
2. **Zero KPAs.** (No formal KPAs; ad hoc, chaotic management).
3. **Level 2: Repeatable (6 KPAs — Basic Project Management Discipline):**
4. *Requirements Management (RM)*
5. *Software Project Planning (SPP)*
6. *Software Project Tracking and Oversight (SPTO)*
7. *Software Subcontract Management (SSM)*
8. *Software Quality Assurance (SQA)*
9. *Software Configuration Management (SCM)*
10. **Level 3: Defined (7 KPAs — Organization-Wide Standard Process):**
11. *Organization Process Focus (OPF)*
12. *Organization Process Definition (OPD)*
13. *Training Program (TP)*
14. *Integrated Software Management (ISM)*
15. *Software Product Engineering (SPE)*
16. *Intergroup Coordination (IC)*
17. *Peer Reviews (PR)*
18. **Level 4: Managed (2 KPAs — Quantitative & Statistical Process Control):**
19. *Quantitative Process Management (QPM)*
20. *Software Quality Management (SQM)*
21. **Level 5: Optimizing (3 KPAs — Continuous Process Improvement):**
22. *Defect Prevention (DP)*
23. *Technology Change Management (TCM)*

24. *Process Change Management (PCM)*

ملاحظة أكاديمية هامة (Academic Note)

توزيع مجالات العمل الرئيسية (KPAs): المستوى 1: (0) ← المستوى 2: (6) ← المستوى 3: (7) ← المستوى 4: (2) ← المستوى 5: (3) ← المجموع الكلي: 18 KPA.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.2, pp. 350–353; companion slides 97–101.

Question 7.18 · Defined Level KPAs & Peer Review Activities (Exercise 7.33)

What are the various key process areas at defined level in CMM? Describe activities associated with one key process area.

The 7 Key Process Areas at Level 3 (Defined) are: 1. Organization Process Focus (OPF) 2. Organization Process Definition (OPD) 3. Training Program (TP) 4. Integrated Software Management (ISM) 5. Software Product Engineering (SPE) 6. Intergroup Coordination (IC) 7. Peer Reviews (PR)

Peer Reviews (PR) focus on removing defects from software work products early and efficiently. They also help participants understand the work products and the defects that can be prevented.

ملاحظة أكاديمية هامة (Academic Note)

فكرة Peer Reviews: مراجعة مبكرة لمخرجات العمل تساعد على كشف العيوب وفهمها قبل انتقالها إلى مراحل لاحقة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.2, p. 351; companion slides 98–100.

ASE CHAPTER 7

T3 Comparison & Analytical Breakdown Pack

Comparison 7.1 · Hardware Reliability vs Software Reliability (Exercise 7.3)

Compare hardware reliability with software reliability. Why does software not experience physical wear-out?

While both hardware and software reliability deal with the probability of failure-free operation, they differ fundamentally in failure causes, degradation mechanisms, and maintenance impacts:

- **Nature of Failures:**
- **Hardware failures are predominantly physical:** Caused by physical wear-out, chemical degradation, manufacturing defects, thermal stress, and environmental fatigue.
- **Software failures are strictly design-related:** Software is a logical entity. Software failures occur solely due to **unmanifested design, logic, specification, and coding faults** activated by specific operational input sequences.
- **Failure Trajectory Over Time:**
- **Hardware follows the classic Bathtub Curve:** Characterized by high burn-in infant mortality, an extended constant-hazard operational phase, and a sharp upward wear-out period due to physical degradation.
- **Software does not wear out:** Software possesses no physical moving parts, does not rust, and suffers zero wear-out. **Ideally, software failure rate decreases monotonically over time as faults are identified and fixed.** In practice, software failure intensity exhibits sudden upward spikes whenever code changes or maintenance activities introduce regression faults, followed by subsequent burn-in.
- **Effect of Maintenance and Repairs:**
- **Hardware repair restores original reliability:** Replacing a worn-out component with a fresh spare part restores the machine to its previous operating baseline.
- **Software "repair" changes system design:** Software maintenance does not replace identical physical parts; **it alters the code logic itself.** A software bug fix can introduce new subtle faults, altering the program structure and potentially lowering overall system reliability.
- **Manufacturing and Duplication:**
- Hardware reliability varies between units due to manufacturing variances and tolerances.
- Software duplication is bit-for-bit exact; if one copy of a software release has a latent defect, every identical copy contains that exact same defect.

ملاحظة أكاديمية هامة (Academic Note)

الفارق الجوهري: العتاد يبلى فيزيائياً مع الزمن (Wear-out) واستبدال القطعة يعيده لحالته الأصلية ← البرمجيات لا تبلى، لكن "إصلاحها" هو تعديل تصميمي قد يدخل عيوباً جديدة (Regression faults) تغير هيكل النظام.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.1, pp. 308–310; companion slides 2–5.

Comparison 7.2 · Software Quality vs Software Reliability (Exercise 7.22)

Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss them.

Although software quality and software reliability are closely linked quality attributes, they differ in conceptual breadth, operational measurement, and time dependency:

- **Conceptual Scope (Umbrella vs Component):**
- **Software Quality is an overarching, multi-dimensional umbrella concept:** It encompasses the totality of attributes that satisfy customer requirements and business needs, including Functionality, Usability, Efficiency, Maintainability, Portability, and Reliability.
- **Software Reliability is a specific, focused attribute of quality:** It focuses strictly on failure-free operational execution over specified time intervals.
- **Time Dependency:**
- **Quality can be assessed statically without execution:** Aspects of quality such as code readability, modularity, architectural documentation, and structural complexity exist and can be measured independently of runtime.
- **Reliability is strictly dynamic and time-dependent:** Reliability cannot exist in the abstract without execution time or operational usage; it is fundamentally a probability defined over an operational time interval ($[0, t]$).
- **Measurement Metric:**
- **Quality is assessed through diverse qualitative and quantitative criteria:** Function points, cyclomatic complexity, usability surveys, and compliance audits.
- **Reliability is evaluated through precise stochastic and statistical indicators:** Failure intensity (λ), Mean Time Between Failures (MTBF), Mean Time To Failure (MTTF), and cumulative failure distributions ($\mu(t)$).
- **Operational Independence:**
- A software system can be mathematically reliable (never crashes because it executes a stable loop) while having poor quality (terrible user interface, high computational resource consumption, impossible to modify).
- Conversely, software may possess high internal quality (modular, well-documented, clean architecture) but suffer low operational reliability if key boundary-condition bugs crash the system under heavy operational load.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

خلاصة المقارنة: الجودة مفهوم كلي يشمل الموثوقية ← يمكن قياس أبعاد الجودة استاتيكياً (مثل سهولة القراءة والتعقيد)، بينما الموثوقية مرتبطة حصراً بالتشغيل الديناميكي والزمن والاحتمالية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2, pp. 321–323; companion slides 24–29.

Comparison 7.3 · ISO 9126 vs McCall Software Quality Model (Exercise 7.14)

Compare ISO 9126 with the McCall software quality model and highlight the key advantages of ISO 9126.

Both McCall (1977) and ISO 9126 (1991) establish hierarchical models to evaluate software product quality, but they exhibit structural and methodological differences:

- **Historical Context & Origin:**
- **McCall Model:** Developed for the US Air Force and military procurement; primarily oriented toward defense contracts and traditional waterfall lifecycles.
- **ISO 9126:** Developed by international consensus through the International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) as a universal commercial standard.
- **Top-Level Structural Organization:**
- **McCall organizes 11 factors into 3 operational perspectives:** Product Operation (5 factors), Product Revision (3 factors), and Product Transition (3 factors).
- **ISO 9126 organizes quality into 6 distinct characteristics:** Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability.
- **Decomposition Depth:**
- McCall presents 11 quality factors and 25 quality criteria.
- ISO 9126 decomposes 6 characteristics into 21 clearly delineated sub-characteristics (attributes), establishing a clean boundary between external user-visible behaviors and internal code properties.

Key Advantages of ISO 9126 over the McCall Model:

1. **Elimination of Factor Overlap:** In McCall's model, criteria like *Simplicity* and *Consistency* overlap across multiple factors. ISO 9126 establishes strict non-overlapping boundaries among characteristics to ensure orthogonal evaluation.
2. **Explicit Functionality Characteristic:** McCall distributes functional aspects across correctness and integrity without an explicit top-level characteristic. ISO 9126 explicitly elevates **Functionality** (Suitability, Accuracy, Interoperability, Compliance, Security) as a primary pillar of quality.
3. **International Standardization and Harmonization:** ISO 9126 represents broad global consensus across industries, whereas McCall represents early US defense procurement guidelines.
4. **Modern Lifecycle Adaptability:** ISO 9126 cleanly distinguishes between process quality, internal product metrics, external product metrics, and quality-in-use.

ملاحظة أكاديمية هامة (Academic Note)

مزايا ISO 9126 على مكول: 1. جعل "الوظائفية" (Functionality) خاصية عليا مستقلة.
2. إزالة التداخل المعياري بين العوامل وتحقيق التعامد (Orthogonality).
3. اعتماد عالمي موحد متوافق مع مختلف قطاعات الصناعة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1, Fig. 7.10, p. 322; Table 7.5(b), pp. 323–324; §7.2.4, pp. 326–328; companion slides 35–46.

Comparison 7.4 · Boehm Software Quality Model vs McCall Model

Compare Barry Boehm's software quality model with the McCall software quality model.

Both Boehm and McCall organize software-quality attributes hierarchically, but their model structures differ:

- **Top-Level Perspective:**
- **McCall's Top Level:** Focuses on **operational perspectives of software use and maintenance** (Operation, Revision, Transition).
- **Boehm's Top Level:** General Utility links directly to **Portability** and **Maintainability**. As-is Utility links to Reliability, Efficiency, and Human Engineering; Maintainability links to Testability, Understandability, and Modifiability.
- **Hierarchical Level Decomposition:**
- **McCall Model:** Uses a 3-tier hierarchy: **Quality Factors** → **Quality Criteria** → **Metrics**.
- **Boehm Model:** The seven intermediate constructs are **Portability, Reliability, Efficiency, Human Engineering, Testability, Understandability, and Modifiability**. The model lists 12 primitive characteristics, including device independence, completeness, accuracy, consistency, device efficiency, accessibility, communicativeness, self-descriptiveness, structuredness, conciseness, legibility, and augmentability.
- McCall's model organizes 11 quality factors under the three product perspectives and further relates them to criteria and metrics.

ملاحظة أكاديمية هامة (Academic Note)

الفارق المعماري: يربط الشكل General Utility مباشرةً بـ Portability وMaintainability؛ ويربط As-is Utility بـ Reliability وEfficiency وHuman Engineering، وMaintainability بـ Testability وUnderstandability وModifiability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1–§7.2.2, pp. 324–326; companion slides 40–41.

Comparison 7.5 · Basic Execution Time Model vs Logarithmic Poisson Execution Time Model (Exercise 7.17)

Explain the Basic and Logarithmic Poisson software reliability models and their significance in reliability studies.

The Basic Execution Time Model and the Logarithmic Poisson Execution Time Model are execution-time reliability growth models with different assumptions and failure-intensity functions:

- **Failure Intensity vs Failures Experienced:**
- **Basic Model: Linear relationship.** Failure intensity decreases linearly with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

$$\frac{d\lambda}{d\mu} = -\frac{\lambda_0}{v_0}$$

The model gives a constant linear decrease in failure intensity per failure experienced.

- **Logarithmic Poisson Model: Exponential relationship.** Failure intensity decreases exponentially with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

The absolute reduction per additional failure becomes smaller as failure intensity falls, because the reduction is proportional to the current intensity.

- **Total Failures Over Infinite Time (v_0):**
- **Basic Model: Finite.** Assumes a finite total expected number of failures v_0 over unbounded execution time.
- **Logarithmic Poisson Model: Infinite.** Its model form has an infinite expected number of failures over unbounded execution time ($v_0 = \infty$).
- **Failure Intensity vs Execution Time (τ):**
- **Basic Model:** Decays **exponentially** with execution time:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

- **Logarithmic Poisson Model:** Decays **inversely** with execution time:

$$\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}$$

- **Significance in Reliability Studies:**
- The models represent different assumptions about failure-intensity reduction and the total expected number of failures. Which one is more suitable depends on how well its assumptions match the system and observed testing data; neither is universally superior.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة محورية: - الأساسي (Basic): تناقص خطي مع الأخطاء (μ)، وعدد الأخطاء الكلي محدود ($v_0 < \infty$). - اللوغاريتمي (Logarithmic): تناقص أسي مع الأخطاء (μ)، وعدد الأخطاء عبر الزمن اللانهائي غير محدود ($v_0 = \infty$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1 & §7.3.2, pp. 329–339; companion slides 47–71.

Comparison 7.6 · Jelinski–Moranda Model vs Musa Basic Execution Time Model

Compare the Jelinski–Moranda reliability model with Musa's Basic Execution Time Model.

Both Jelinski–Moranda and Musa Basic are finite-failure reliability growth models, but they represent the failure process differently:

- **Time Representation — Jelinski–Moranda:** Represents the hazard for each interval between successive failure events; the model's hazard changes after a failure is removed.
- **Time Representation — Musa Basic Model:** Defines failure intensity against execution time (τ, CPU time) and failures experienced. A separate calendar-time component can relate execution time to elapsed calendar time.
- **Hazard Function Structure:**
- **Jelinski–Moranda:** Defines a **discrete, piecewise constant hazard rate** during the inter-arrival interval between the $(i-1)$-th and i-th failure:

$$\lambda(t_i) = \phi[N - (i-1)]$$

  The hazard rate graph is a discrete descending staircase.
- **Musa Basic Model:** Formulates failure intensity as a **continuous function of execution time** and cumulative failures experienced:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right), \quad \lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{v_0}\right)$$

- **Fault-to-Failure Relationship:** Jelinski–Moranda assumes one fault is removed after each observed failure. Musa Basic represents progress through failures experienced (μ) and the finite expected-failure parameter (v_0).

📝 **ملاحظة أكاديمية هامة (Academic Note)**

وجه الشبه والاختلاف: كلا النموذجين يفترضان عدداً محدوداً من الأخطاء وتناقصاً ثابتاً لشدة الفشل مع إزالة العيوب ← نموذج Jelinski–Moranda يغيّر الشدة على خطوات بعد كل فشل، بينما Musa Basic يعبّر عنها بدلالة زمن التنفيذ وعدد حالات الفشل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1 & §7.3.4, pp. 329–334, 343–346; companion slides 47–58, 89–92.

Comparison 7.7 · Mills Bug Seeding vs Software Reliability Growth Models (SRGMs)

Compare the Mills bug seeding technique with analytical Software Reliability Growth Models (such as Musa and Jelinski–Moranda).

Bug seeding estimates a remaining fault count from discovery counts, whereas software reliability growth models describe failure behavior over execution time:

- **Input Data Required:**
- **Bug Seeding:** Uses the number of seeded faults (S), detected seeded faults (s), and detected indigenous faults (n). It does not use elapsed execution time in the basic estimate.
- **SRGMs:** Use observed failure counts and execution-time or failure-time data, depending on the model.
- **Output Metrics Provided:**

- **Bug Seeding:** Estimates a total indigenous-fault count ($\hat{N}$), from which an estimated remaining count can be obtained. The estimate is not a failure-intensity forecast.
- **SRGMs:** Estimate or predict failure intensity and reliability-growth behavior as testing proceeds.
- **Practical Vulnerabilities:**
- Bug seeding assumes seeded and indigenous bugs have comparable chances of detection. If seeded bugs are easier to find, the estimate can be biased downward.
- Reliability-growth models use model-appropriate failure and time observations. Musa's calendar-time estimates additionally require resource and calendar-time data.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة منهجية: - زراعة الأخطاء (Bug Seeding): تستخدم أعداد الأخطاء المزروعة والمكتشفة لتقدير العدد الكلي للأخطاء الأصلية، ثم المتبقي منها. - نماذج نمو الموثوقية (SRGMs): تربط بيانات الفشل بزمن التنفيذ لتقدير شدة الفشل وسلوك نمو الموثوقية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1–§7.3.5, pp. 329–347; companion slides 47–93.

Comparison 7.8 · SEI CMM vs ISO 9001: Structural Philosophy & Differences (Exercise 7.31 & 7.34)

List the differences between CMM and ISO 9001. Why is it suggested that CMM is the better choice for software organizations?

The two frameworks overlap, but differ in purpose and emphasis:

Scope: ISO 9001 is a general quality-management standard for many industries. CMM is focused on software development; it can be applied to development and maintenance, although it does not directly address every maintenance-specific concern.
Emphasis: ISO 9001 sets minimum criteria for an adequate quality-management system. CMM places stronger emphasis on continuous software-process improvement and offers more detailed guidance for software organizations.
Shared principle: Both emphasize documenting the process and following the documented process.
Practical choice: CMM is the more detailed guide when the goal is software-process improvement. ISO 9001 certification may still be needed when a customer or market requires it. The goal should be process improvement, not the certificate or maturity score.

Overall, ISO 9001's general concerns are largely encompassed by CMM, while ISO 9001 does not cover all of CMM's software-process-improvement concerns.

ملاحظة أكاديمية هامة (Academic Note)

CMM أدق لتحسين عمليات البرمجيات، وISO 9001 يفيد عند الحاجة إلى شهادة يطلبها السوق أو الزبون. التداخل بينهما لا يعني أنهما متطابقان.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.4–§7.5.3, pp. 352–357; companion slides 97–107.

Comparison 7.9 · ISO 9001 Certification vs SEI CMM-Based Evaluation (Exercise 7.35)

Discuss the relative merits of ISO 9001 certification and the SEI CMM-based evaluation. Point out some of the shortcomings of the ISO 9001 certification process as applied to the software industry.

Relative merits and limits:

ISO 9001: It provides a recognized quality-management certification and a minimum set of criteria for an adequate quality system. A customer or market may require the certificate.
CMM: It focuses on software processes, stresses continuous improvement, and gives more detailed guidance for improving software organizations.
ISO 9001 in software: Its general scope means its requirements need interpretation for software work. It defines an adequate quality system, but does not provide the same detailed software-process-improvement guidance as CMM.
CMM evaluation: A maturity score is useful as a process-improvement reference, but achieving a level should not replace the goal of improving the process itself.

Conclusion: Use CMM when the main objective is detailed software-process improvement, and address ISO 9001 where certification is required. The frameworks overlap substantially, but neither the certificate nor a maturity score is the improvement goal.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ميزة CMM هي التوجيه التفصيلي للتحسين المستمر؛ وقد تبقى شهادة ISO مطلوبة تجارياً. لا تساوي الشهادة مستوى نضج CMM، ولا يُعدّ أيٌّ منهما غاية التحسين.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.4–§7.5.3, pp. 352–357; companion slides 103–107.

Comparison 7.10 · Relationships Between ISO 9001:1994 Clauses and CMM Practices (Exercises 7.29 & 7.30)

Discuss the correlation between the 20 clauses of ISO 9001 (1994) and the Key Process Areas (KPAs) of the SEI Capability Maturity Model.

The relationships are not one-to-one. One ISO clause may involve several CMM practices, and some ISO concerns are outside CMM's scope. The following are the clause-to-practice correspondences:

1. Management responsibility: Primarily Software Quality Assurance (SQA); Software Project Planning (SPP) and Software Project Tracking and Oversight (SPTO) also establish project responsibilities.
2. Quality system: Primarily SQA. Relevant procedures are distributed across Activities Performed practices; SPP specifies project procedures, and SQA and Verifying Implementation check compliance.
3. Contract review: Requirements Management clarifies software requirements allocated from customer needs.
4. Design control: Software Product Engineering (SPE) covers requirements analysis, design, coding, and testing; SPP plans these activities, SPTO tracks them, and Software Configuration Management (SCM) controls their work products.

5. Document control: SCM controls document distribution and modification; specific project procedures and standards are distributed across practices.
6. Purchasing: Software Subcontract Management addresses the supplier relationship.
7. Purchaser-supplied product: Integrated Software Management addresses purchased, off-the-shelf, or reusable software; coverage in this area is weak.
8. Product identification and traceability: Primarily SCM.
9. Process control: Procedures are distributed across CMM practices; project-specific procedures and standards are specified in the software development plan.
10. Inspection and testing: Product testing is in SPE; software-process inspections are addressed by Peer Reviews.
11. Inspection, measuring, and test equipment: Generally addressed through testing practices in SPE.
12. Inspection and test status: Testing practices in SPE address test status; problem reporting and configuration status are addressed through SCM.
13. Control of nonconforming product: Design, implementation, testing, and validation are addressed in SPE; SCM tracks configuration-item status and known unfixed defects. Installation is not addressed.
14. Corrective action: Field-defect analysis and corrective action are handled through problem reporting and controlled maintenance of baseline work products.
15. Handling, storage, packaging, and delivery: CMM does not cover replication, delivery, or installation.
16. Quality records: Record-maintenance practices are distributed across CMM practices.
17. Internal quality audits: SQA addresses the audit process; specific audits are included in the auditing practice of the Verifying Implementation common feature.
18. Training: Training needs and training records are addressed by Training and Orientation practices in the Ability to Perform common feature.
19. Servicing: Maintenance is embedded across CMM practices, but its unique concerns are not directly addressed as a separate process area.
20. Statistical techniques: Measurement practices are distributed across CMM; product measurement appears among Activities Performed practices, while process measurement is addressed by the Measurement and Analysis common feature.

Overall, the frameworks overlap substantially, but do not form an exact route map. ISO 9001's general quality-system concerns are largely encompassed by CMM; the reverse is less true. Some specific ISO concerns remain outside CMM, so the frameworks should not be treated as equivalent.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **العلاقة تقريبية وليست بنداً مقابل بند. بعض المتطلبات موزعة بين أكثر من ممارسة، وبعضها مثل التسليم لا يغطيه CMM؛ لذلك لا يصح تحويل التداخل إلى نسبة امتثال أو مساواة كاملة.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.5.1–§7.5.3, pp. 353–357; companion slides 103–107.

ASE CHAPTER 7

T4 Visual Models & Diagram Interpretation Pack

Q7.V01 · Hardware Bathtub Curve and Software Reliability Curve (Exercise 7.2)

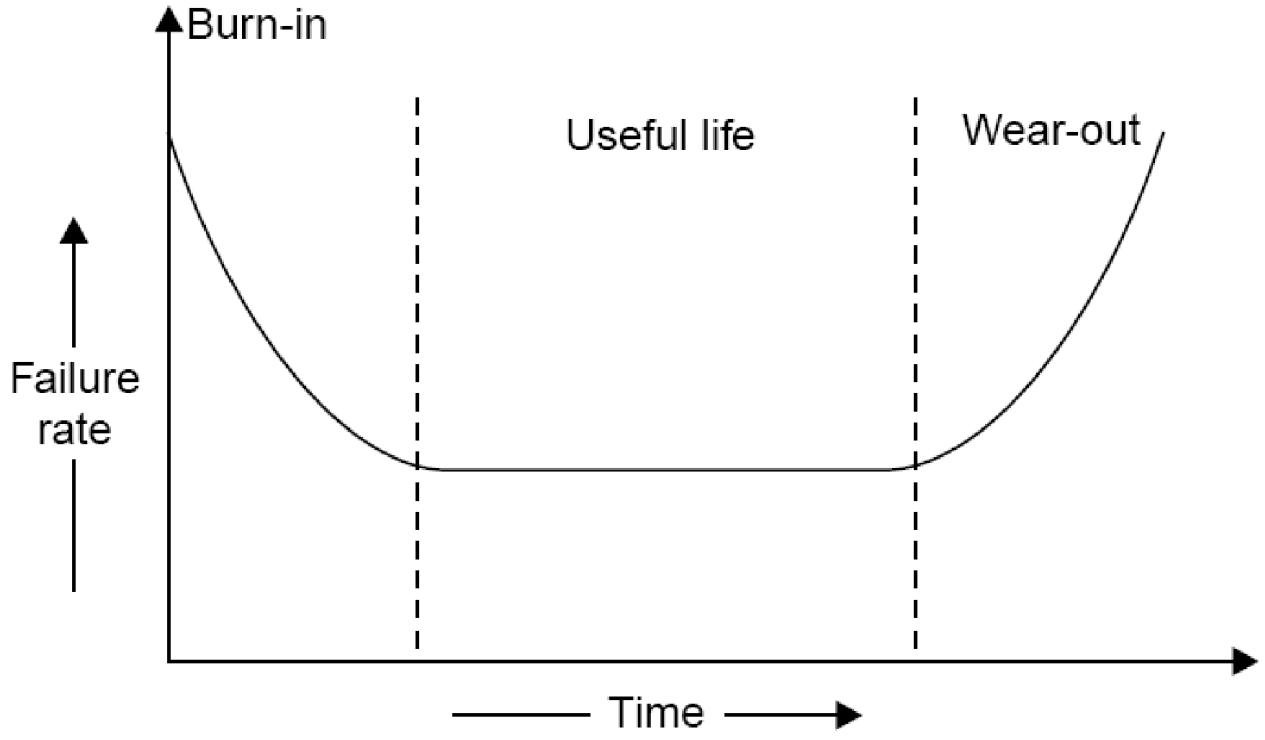


Fig. 7.1: Bath tub curve of hardware reliability.

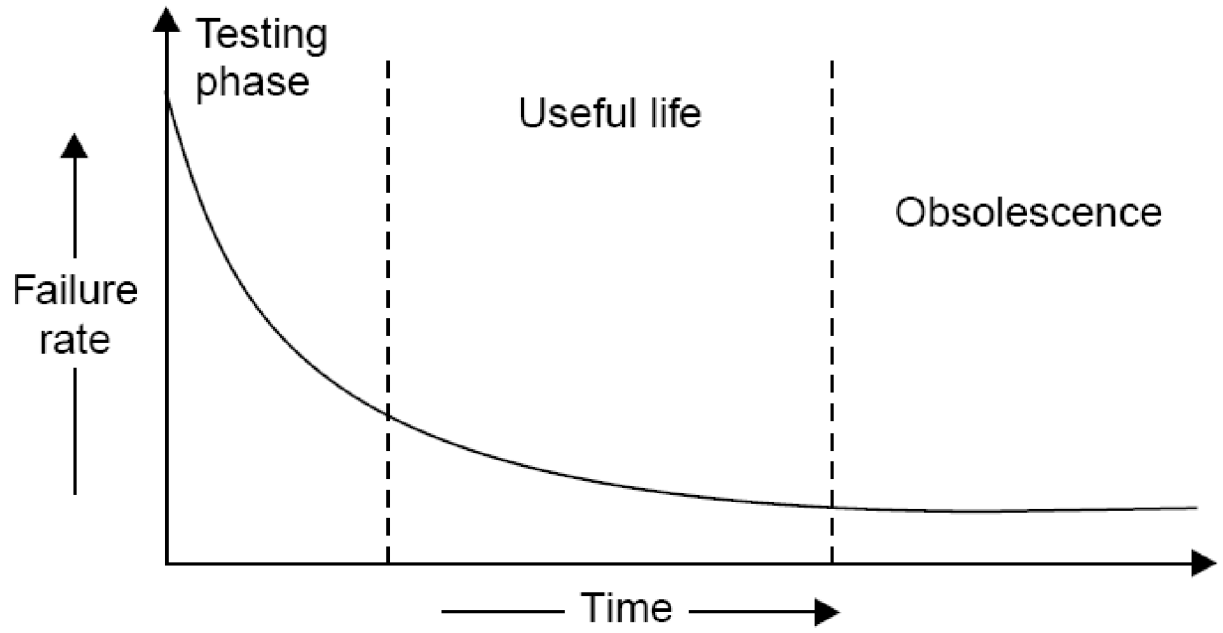


Fig. 7.2: Software reliability curve (failure rate versus time)

1. **Figure 7.1:** Hardware failure rate falls during burn-in, stays nearly constant through useful life, then rises during wear-out.
2. **Figure 7.2:** Software failure rate falls during testing, remains low through useful life, and reaches the obsolescence phase. This plotted curve has no hardware-style wear-out phase.

Academic Note

ملاحظة المخطط: يوضح الشكل الأول ارتفاع فشل العتاد بعد مرحلة العمر النافع، بينما يوضح الثاني انخفاض فشل البرمجيات أثناء الاختبار واستقراره خلال العمر النافع ثم تقادمها.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.1, pp. 308–310; companion slides 3–4, Figs. 7.1–7.2.

Q7.V02 · Input Space and Example Input-State Probabilities (Exercise 7.6)

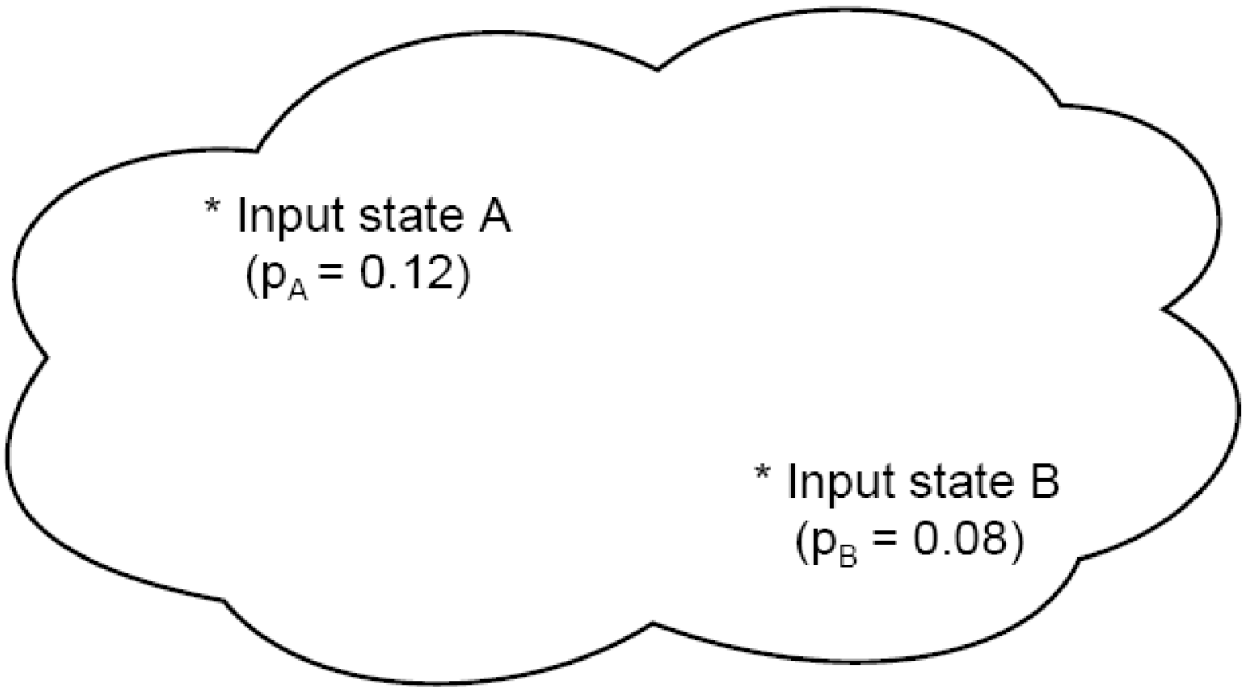


Fig. 7.4: Input Space

- The cloud labelled **Input Space** shows two example states: A with probability 0.12 and B with probability 0.08.
- These labels associate occurrence probabilities with the two states; the figure does not mark a failure domain.

Academic Note

ملاحظة المخطط: الشكل يعرض حالتي إدخال واحتماليهما؛ لا يرسم منطقة فشل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.4, pp. 314–316; companion slide 18, Fig. 7.4.

Q7.V03 · Probability Portions in an Operational Profile (Exercise 7.6)

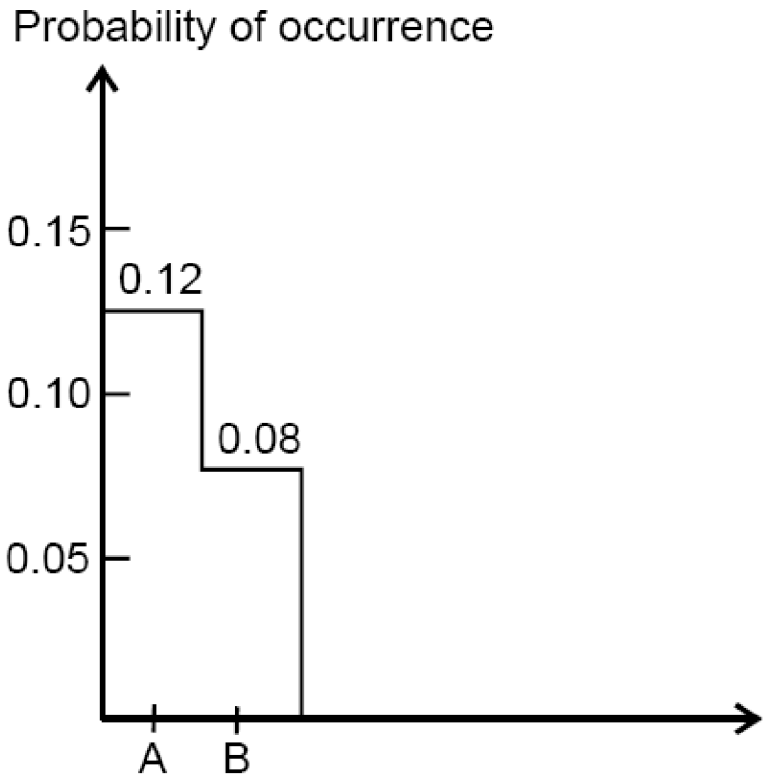


Fig. 7.5: Portion of operational profile

- The vertical axis is probability of occurrence; the input states A and B are shown with probabilities 0.12 and 0.08.
- The caption identifies this as a portion of an operational profile, not a complete branching tree.

Academic Note

ملاحظة المخطط: يبين الجزء المعروض احتمال ظهور الحالة A والحالة B؛ لا يعرض شجرة تشغيلية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.5, pp. 316–318; companion slide 19, Fig. 7.5.

Q7.V04 · McCall Quality Factors and Criteria (Exercises 7.10 & 7.11)

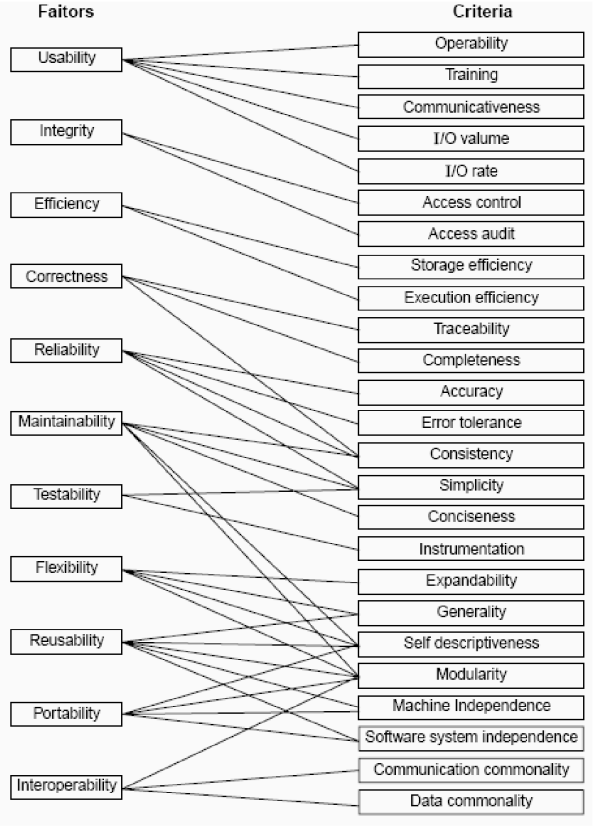


Fig 7.10: McCall's quality model

- The figure maps 11 quality factors to 25 quality criteria. Its connecting lines show which criteria are associated with each factor; a criterion can connect to more than one factor.
- This is a factor-to-criteria map; it does not show the separate Product Operation, Product Revision, and Product Transition grouping.

Academic Note

ملاحظة المخطط: يربط الشكل 11 عاملاً للجودة بـ25 معياراً؛ تجميع Operation وRevision وTransition يخص شكلاً آخر.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1, p. 322; companion slide 35, Fig. 7.10.

Q7.V05 · Boehm Software Quality Model (Exercise 7.12)

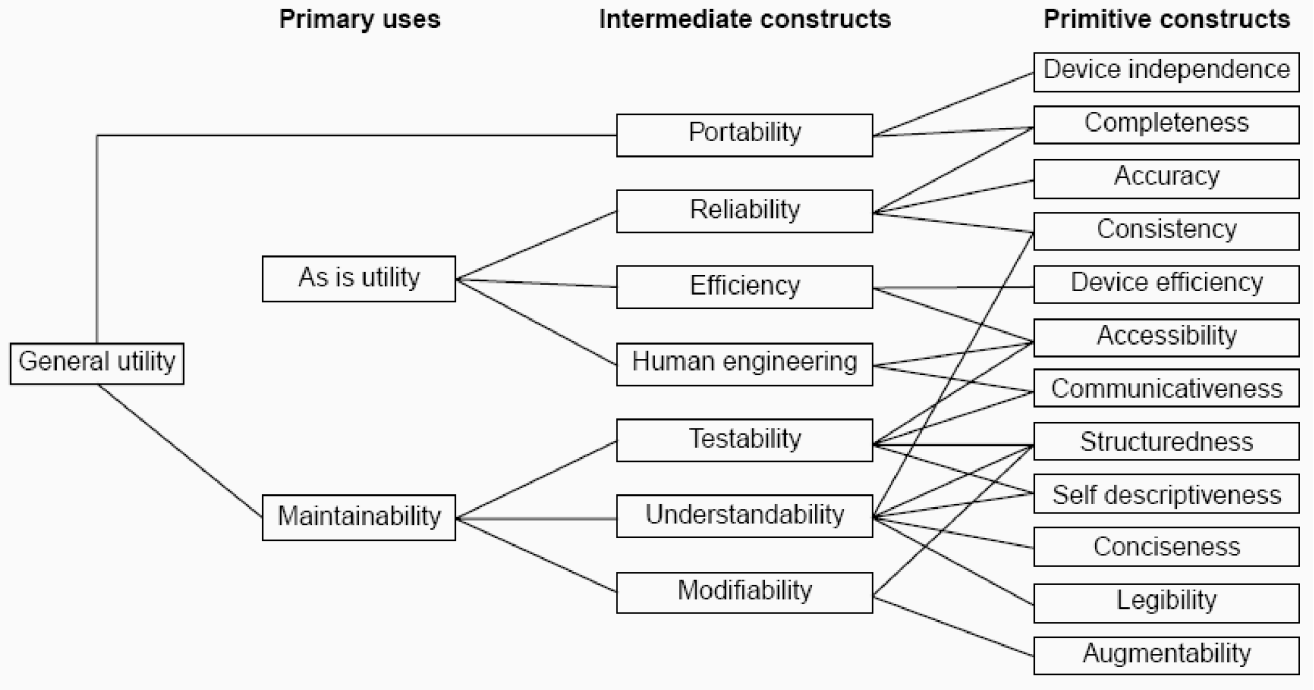


Fig.7.11: The Boehm software quality model

- The figure shows three levels: primary uses, intermediate constructs, and primitive constructs.
- General Utility links directly to Portability and Maintainability. As-is Utility links to Reliability, Efficiency, and Human Engineering; Maintainability links to Testability, Understandability, and Modifiability.
- The figure contains 7 intermediate constructs and 12 primitive constructs.

Academic Note

ملاحظة المخطط: يتدرج نموذج Boehm من الاستخدامات إلى التراكيب الوسيطة ثم الخصائص الأولية؛ يضم الشكل 7 تراكيب وسيطة و12 خاصية أولية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.2, pp. 324–325; companion slide 40, Fig. 7.11.

Q7.V06 · ISO 9126 Quality Model (Exercise 7.13)

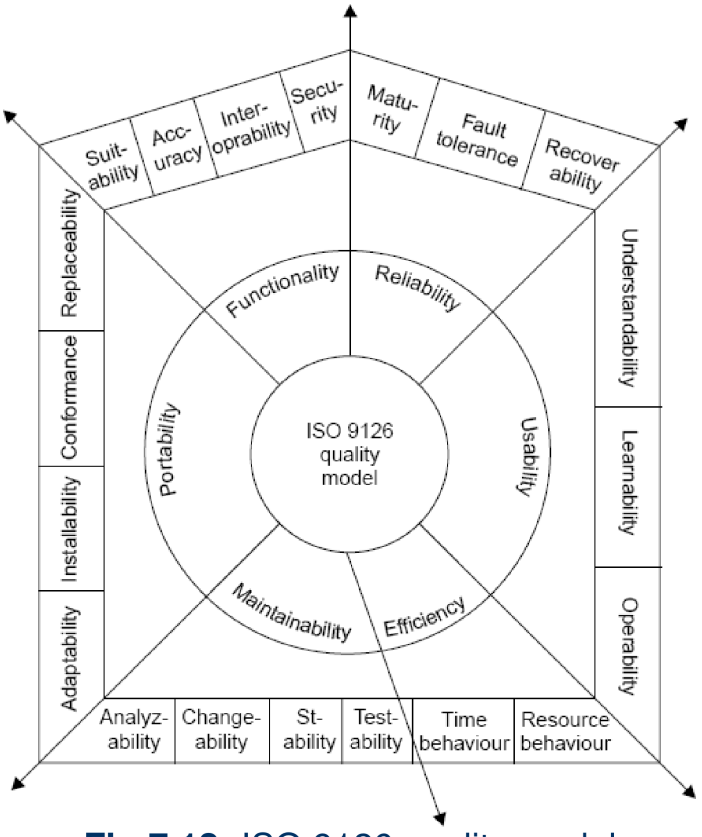


Fig.7.12: ISO 9126 quality model

The figure groups 20 subcharacteristics under six characteristics:

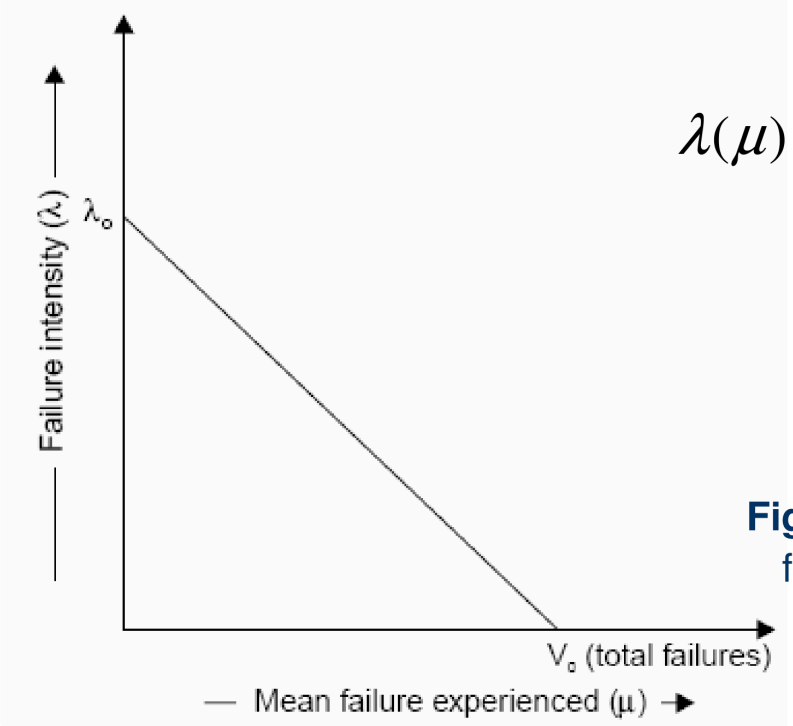
1. **Functionality:** Suitability, Accuracy, Interoperability, Security.
2. **Reliability:** Maturity, Fault tolerance, Recoverability.
3. **Usability:** Understandability, Learnability, Operability.
4. **Efficiency:** Time behaviour, Resource behaviour.
5. **Maintainability:** Analyzability, Changeability, Stability, Testability.
6. **Portability:** Adaptability, Installability, Conformance, Replaceability.

Academic Note

ملاحظة المخطط: الشكل يعرض 6 خصائص رئيسية و20 فرعية؛ Conformance تابعة لـPortability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.3, pp. 326–328; companion slide 46, Fig. 7.12.

Q7.V07 · Basic and Logarithmic Poisson Failure-Intensity Curves (Exercises 7.15 & 7.17)



$$\lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{V_0}\right) \quad (1)$$

Fig.7.13: Failure intensity λ as a function of μ for basic model

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

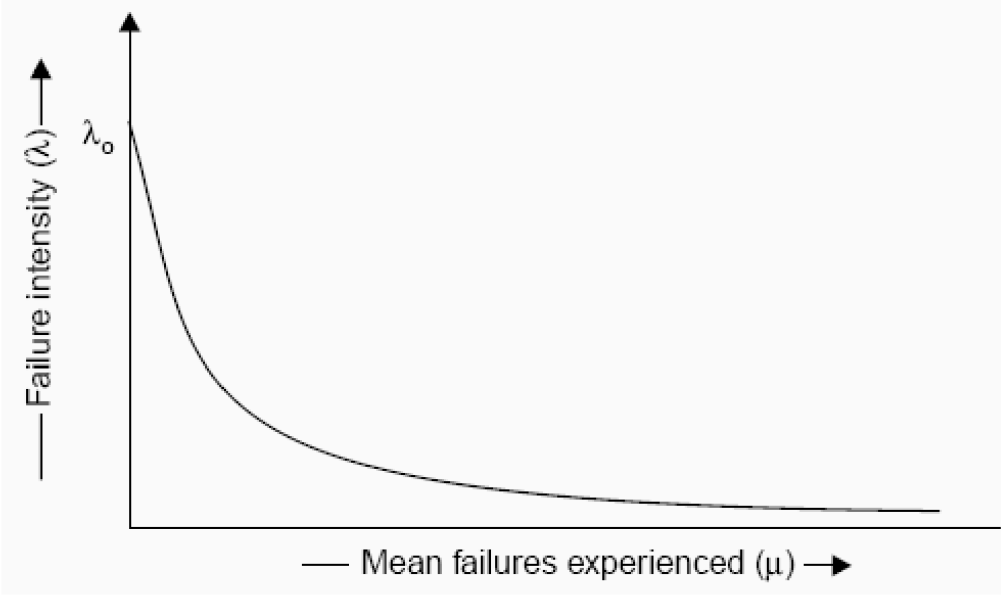


Fig.7.18: Relationship between μ & λ

1. **Basic model, Figure 7.13:** Plots failure intensity λ against mean failures experienced μ. The straight line falls from λ_0 at $\mu = 0$ to zero at $\mu = V_0$; the displayed equation is $\lambda(\mu) = \lambda_0(1 - \mu/V_0)$.
2. **Logarithmic Poisson model, Figure 7.18:** Also plots λ against μ, using $\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$. Its curve falls steeply at first, then gradually flattens toward zero.

Academic Note

ملاحظة المخطط: الشكلان كلاهما λ مقابل μ: الأساسي خط مستقيم، واللوغاريتمي منحنى يتناقص ثم يتسطح.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1, p. 330, and §7.3.2, p. 336; companion slides 47 and 59, Figs. 7.13 and 7.18.

Q7.V08 · Calendar-Time to Execution-Time Ratio and Limiting Resources (Exercise 7.23)

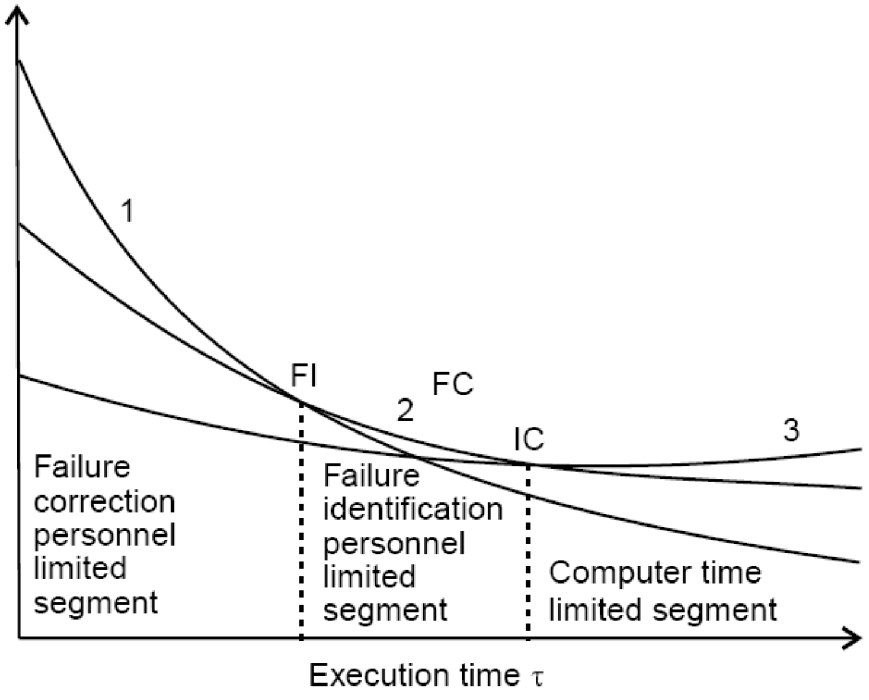


Fig.7.21: Calendar time to execution time ratio for different limiting resources

- The plot relates the calendar-time/execution-time ratio to execution time τ.
- It labels three resource-limited segments: failure-correction personnel, failure-identification personnel, and computer time.

> **Academic Note**
> **ملاحظة المخطط:** القطاعات الثلاثة المعلّمة هي محدودية موظفي التصحيح، ثم موظفي تحديد الفشل، ثم وقت الحاسوب.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.3, pp. 340–342; companion slide 77, Fig. 7.21.

Q7.V09 · Jelinski–Moranda Failure-Intensity Steps (Exercise 7.25)

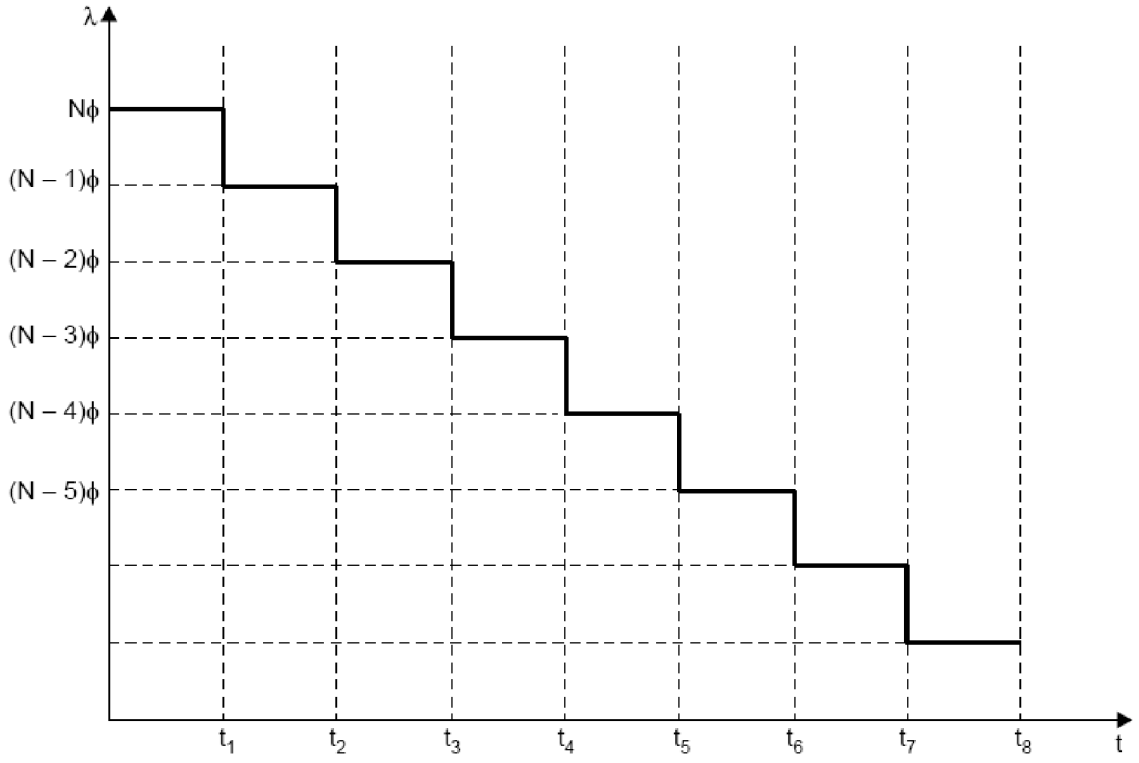


Fig.7.22: Relation between t & λ

- The plot shows failure intensity λ against time t. The rate remains constant between failures and drops by φ at successive failure times, from Nφ to (N − 1)φ and onward.

> **Academic Note**
> **ملاحظة المخطط:** يبقى λ ثابتاً بين أزمنة الفشل، ثم ينخفض بمقدار φ عند كل خطوة لاحقة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.4, pp. 343–345; companion slide 90, Fig. 7.22.

Q7.V10 · SEI CMM Maturity Levels and Characterizations (Exercises 7.27 & 7.28)

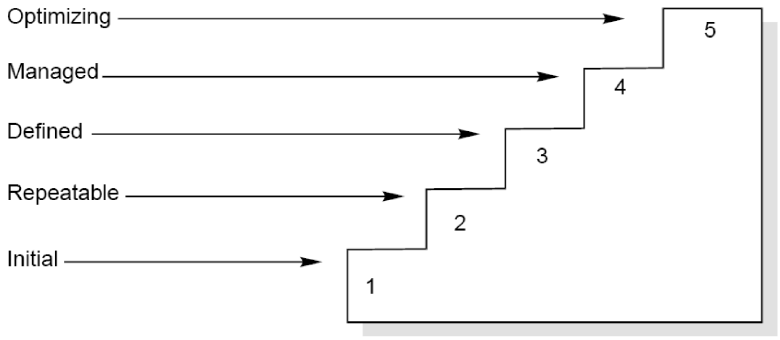


Fig.7.23: Maturity levels of CMM

Maturity Level	Characterization
Initial	Adhoc Process
Repeatable	Basic Project Management
Defined	Process Definition
Managed	Process Measurement
Optimizing	Process Control

Fig.7.24: The five levels of CMM

- Figure 7.23 orders the five levels: Initial (1), Repeatable (2), Defined (3), Managed (4), and Optimizing (5).
- Figure 7.24 pairs them with these characterizations: Adhoc Process, Basic Project Management, Process Definition, Process Measurement, and Process Control, respectively.

Academic Note

ملاحظة المخطط: الشكل الأول يبيّن ترتيب المستويات الخمسة، والثاني يربط كل مستوى بوصفه المقابل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.1, pp. 347–350; companion slides 94 and 96, Figs. 7.23–7.24.

ASE CHAPTER 7

T5 Applied Scenarios & Interpretation Pack

Scenario 7.1 · Operational Profile Construction & Test Case Allocation

An enterprise software team is developing a high-throughput automated banking system handling four core customer transactions: *Cash Withdrawal*, *Fund Transfer*, *Balance Inquiry*, and *Bill Payment*. Historical production data indicates that out of 500,000 daily transactions, 250,000 are Cash Withdrawals, 125,000 are Balance Inquiries, 75,000 are Fund Transfers, and 50,000 are Bill Payments.

1. Construct the quantitative operational profile for this banking application.
2. If the quality assurance budget permits running a suite of 20,000 automated operational test cases before deployment, calculate how test cases should be distributed across the four transaction types.
3. Explain how testing in accordance with this operational profile may affect reliability growth compared with equal test distribution.

1. Quantitative Operational Profile Construction:

- Total daily transactions $N = 500,000$.
- Probability of Cash Withdrawal:

$$P(\text{Withdrawal}) = \frac{250,000}{500,000} = 0.50 \quad (50\%)$$

- Probability of Balance Inquiry:

$$P(\text{Inquiry}) = \frac{125,000}{500,000} = 0.25 \quad (25\%)$$

- Probability of Fund Transfer:

$$P(\text{Transfer}) = \frac{75,000}{500,000} = 0.15 \quad (15\%)$$

- Probability of Bill Payment:

$$P(\text{Payment}) = \frac{50,000}{500,000} = 0.10 \quad (10\%)$$

- Sum of probabilities: $0.50 + 0.25 + 0.15 + 0.10 = 1.00$.

2. Distribution of 20,000 Automated Test Cases:

- **Cash Withdrawal:** $20,000 \times 0.50 = 10,000 \text{ test cases}$
- **Balance Inquiry:** $20,000 \times 0.25 = 5,000 \text{ test cases}$
- **Fund Transfer:** $20,000 \times 0.15 = 3,000 \text{ test cases}$
- **Bill Payment:** $20,000 \times 0.10 = 2,000 \text{ test cases}$

3. Reliability Growth Acceleration Rationale:

- **Faults residing in frequently executed code paths have a vastly higher probability of activation during live operation.**
- If tests are distributed uniformly ($5,000$ tests per transaction), Cash Withdrawal (which represents 50% of observed transactions) would be severely undertested, leaving high-exposure defects dormant until release.

- Allocating 50% of the test cases to Cash Withdrawal aligns the test distribution with observed use. This can expose faults in frequently used transaction paths earlier, but it does not guarantee the fastest possible reliability growth.

> 📝 **Academic Note**
>
> **القاعدة الهندسية:** توزيع الاختبار وفق البروفايل التشغيلي يطابق كثافة الاختبارات مع الاستخدام المرصود ← وقد يساعد على كشف أخطاء المسارات الشائعة مبكراً، لكنه لا يضمن أسرع نمو ممكن للموثوقية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.5, pp. 316–318; companion slides 16–20.

Scenario 7.2 · Objective-Driven Software Release Certification

A flight-navigation software package is undergoing system testing. The project contract specifies a mandatory release reliability requirement: **Failure intensity must not exceed** $\lambda_F = 0.20\ \text{failure/CPU hr}$. The software development records an initial failure intensity $\lambda_0 = 8.0\ \text{failures/CPU hr}$ and total expected failures $v_0 = 160\ \text{failures}$ under the Basic Execution Time Model. Testing has currently executed for 40 CPU hours and experienced 110 failures.

1. Determine the current failure intensity λ_P and evaluate whether the system currently satisfies the contractual release criterion.
2. Calculate the additional number of failures that must be discovered and resolved, and the additional CPU execution hours required to achieve contractual release readiness.
3. What engineering risks arise if management forces early deployment based on calendar deadline rather than the quantitative failure intensity objective?

1. Current Failure Intensity Evaluation:

- Current failures experienced: $\mu_P = 110\ \text{failures}$.
- Using the Basic Execution Time Model:

$$\lambda_P = \lambda_0 \left(1 - \frac{\mu_P}{v_0}\right) = 8.0 \times \left(1 - \frac{110}{160}\right) = 8.0 \times (1 - 0.6875) = 8.0 \times 0.3125 = 2.50\ \text{failures/CPU hr}$$

- **Release Certification Decision: The software CANNOT be released.** Current failure intensity ($\lambda_P = 2.50\ \text{failures/CPU hr}$) is more than **12 times higher** than the contractual safety threshold ($\lambda_F = 0.20\ \text{failure/CPU hr}$).

2. Additional Testing Effort Required:

- **Additional failures to experience ($\Delta\mu$):**

$$\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F) = \frac{160}{8.0}(2.50 - 0.20) = 20 \times 2.30 = 46\ \text{failures}$$

  (Total failures at release: $\mu_F = 160(1 - 0.20/8.0) = 160 \times 0.975 = 156$ failures; $\Delta\mu = 156 - 110 = 46$).

- **Additional execution time required ($\Delta\tau$):**

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right) = \frac{160}{8.0} \ln\left(\frac{2.50}{0.20}\right) = 20 \times \ln(12.5) = 20 \times 2.525729 \approx 50.51\ \text{CPU hrs}$$

3. Engineering Risks of Premature Release:

- The computed failure intensity is above the contract's stated release limit, so the software does not meet the given release criterion. The scenario does not establish the physical consequences of a failure or any post-deployment cost multiplier.

📝 **Academic Note**

قرار الإطلاق: البرنامج لا يحقق شرط الإطلاق المعطى لأن شدة الفشل المحسوبة أعلى من الحد التعاقدي ← يلزم وفق الحساب اكتشاف 46 فشلاً إضافياً وتنفيذ نحو 50.51 ساعة معالج إضافية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.8 & §7.3.1, pp. 320–322, 329–334; companion slides 21–23, 47–58.

Scenario 7.3 · Environmental Shift & Input Space Activation

A commercial logistics inventory software product was tested rigorously in the software vendor's laboratory for 6 months, exhibiting zero failures across 500 consecutive test hours ($MTTF > 500$ hrs). However, within 48 hours of installation at a customer warehouse in a foreign country, the system crashed multiple times per day.

1. Using the concepts of **Input Space (I)**, **Failure Domain (I_F)**, and **Operational Profile**, explain how software can be failure-free in the laboratory yet fail immediately in the field.
2. Identify the technical flaws in the vendor's testing strategy that permitted this discrepancy.
3. Formulate three concrete corrective actions to prevent environmental failure discrepancies in future releases.

1. Possible Explanation Using Input Space and Operational Profile:

- A latent fault may be activated only by some inputs in the system's input space I, forming a failure domain $I_F \subset I$.
- The zero observed failures in 500 laboratory hours do not prove that the laboratory inputs were disjoint from I_F, nor do they establish 100% reliability. They show only that no failure was observed during the stated test period.
- A difference between laboratory and field operational profiles may have exposed inputs that activate a fault. Locale-specific text, date or currency formats, concurrency, and operating-system settings are possible investigation areas; the scenario provides no evidence that any one of them caused these failures.

2. Flaws in the Vendor's Testing Strategy:

- The scenario does not describe the test inputs or environment, so a specific testing flaw cannot be confirmed. A likely gap to investigate is whether the tests represented the customer's operational profile and environment.
- Review the laboratory test data, field logs, configurations, and input distributions to determine which conditions differ and whether those conditions activate a latent fault.

3. Corrective Engineering Actions:

- **Field Telemetry and Log Analysis:** Capture real customer transaction traces and environmental parameters to build representative operational profiles.
- **Beta Testing and Pilot Field Deployments:** Deploy release candidates in customer staging environments before commercial release.
- **Robust Input Boundary Validation:** Implement defensive programming at all external input interfaces to reject or sanitize inputs outside specified tolerances before they propagate into business logic.

> 📝 **Academic Note**
> **التعليل الهندسي:** قد يفعّل مدخل مغطى بمنطقة الفشل (I_F) عيباً كامناً ← عدم ظهور فشل خلال الاختبار المعملي لا يثبت خلو النظام من الأخطاء، واختلاف بيئة العميل فرضية تحتاج إلى فحص السجلات والبيانات قبل اعتباره سبباً مؤكداً.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.4 & §7.1.5, pp. 314–318; companion slides 16–20.

Scenario 7.4 · SEI CMM Level 2 Readiness Appraisal & Gap Remediation

An independent appraisal assesses a software development organization aiming for SEI CMM Level 2 (Repeatable). The findings reveal the following current organizational practices:

- Project budgets and schedules are estimated informally by project managers based on intuition without historical tracking data.
- System requirements change constantly via verbal phone calls and emails from clients without formal documentation or impact assessments.
- Code source files are stored on developer local hard drives and merged manually via shared network folders without a formal version control tool.
- A dedicated Software Quality Assurance (SQA) group conducts testing at the end of the project, but has no authority to review project plans or audit adherence to development standards.

- Map these audit findings to the 6 Key Process Areas (KPAs) of CMM Level 2 and determine whether the organization qualifies for Level 2 maturity.

- Detail the specific institutional deficiencies across the failed KPAs.
- Formulate an actionable remediation plan outlining practices that would help address the observed gaps before a Level 2 appraisal.

1. KPA Mapping & Appraisal Determination:

- Informal schedule and budget estimation violates **Software Project Planning (SPP)** and **Software Project Tracking and Oversight (SPTO)**.
- Uncontrolled requirements changes violate **Requirements Management (RM)**.
- Manual file merging without tool-enforced versioning violates **Software Configuration Management (SCM)**.
- SQA lacking audit authority over processes and plans violates **Software Quality Assurance (SQA)**.
- **Appraisal conclusion:** The documented gaps are inconsistent with Level 2 practices, so the organization is not ready to demonstrate Level 2 based on this evidence. The scenario gives no information about Software Subcontract Management and is not a complete appraisal; it does not establish the organization's precise current maturity level.

2. Specific Institutional Deficiencies:

- **Absence of Baseline Control (SCM):** High risk of code regression, accidental overwrites, and unverified software releases.
- **Requirements Management (RM):** Uncontrolled requirement changes increase the risk of scope, schedule, and cost deviations.
- **Subjective Project Management (SPP/SPTO):** Lack of disciplined estimation models (e.g., COCOMO, Function Points) or tracking against documented project plans.
- **Weak Quality Governance (SQA):** SQA functions merely as late-stage testing rather than an independent process-compliance auditor reporting directly to senior executive management.

3. Actionable Remediation Plan:

- **Institutionalize Requirements Management:** Establish a formal Change Control Board (CCB) and documented baseline procedure; all requirements modifications must undergo formal impact analysis and customer sign-off before adoption.
- **Establish Formal Project Planning & Tracking:** Adopt standard estimation tools and create documented Software Development Plans (SDPs); conduct bi-weekly formal reviews comparing actual effort, cost, and milestones against the plan.
- **Implement Configuration Management:** Deploy a centralized, secure version control system; establish baseline control for all source code, design documents, and test artifacts.
- **Empower SQA:** Mandate that SQA conduct periodic audits of engineering activities and work products against documented standards, with direct reporting lines to executive management independent of project managers.

📝 **Academic Note**

شروط بلوغ المستوى الثاني (Repeatable): المعطيات تثبت وجود فجوات في مجالات المستوى الثاني، لكنها لا تكفي لتحديد مستوى المؤسسة الحالي أو الحكم على SSM ← يجب استكمال التقييم ومعالجة الفجوات المثبتة قبل تقرير الجاهزية للمستوى الثاني.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.1 & §7.4.2, pp. 347–353; companion slides 94–101.

Scenario 7.5 · Post-Release Maintenance Failure Spikes & Regression Analysis

A cloud enterprise software system achieved its target failure intensity of $\lambda = 0.10\ \text{failure}/\text{CPU hr}$ and was successfully deployed into production. Two months later, the development team released "Service Pack 1" to resolve 15 minor user interface complaints and add a small billing export feature. Within three days of deploying Service Pack 1, production failure intensity suddenly surged to $\lambda = 1.85\ \text{failures}/\text{CPU hr}$, triggering emergency system restarts.

1. Explain this phenomenon using the **Software Reliability Curve** (Figure 7.2) and contrast it with hardware maintenance behavior.
2. Identify the primary engineering cause of this reliability degradation.
3. Formulate a rigorous software maintenance and regression testing protocol to ensure that future maintenance patches do not compromise operational reliability baselines.

1. Reliability Curve Analysis:

- In hardware, maintenance replaces physically worn components with identical spares, reliably restoring the failure rate to its low useful-life baseline.
- In software, **maintenance does not replace physical parts; it alters the logical source code.**
- **Software modifications can introduce regression faults, causing sudden upward spikes in failure intensity.** Here, the increase from 0.10 to $1.85\ \text{failures}/\text{CPU hr}$ illustrates a maintenance-induced failure spike.

2. Plausible Engineering Cause:

- A regression fault introduced by the maintenance change is a plausible explanation for the increase. The timing alone does not identify the faulty change or prove that shared variables, database schemas, or API contracts were involved.
- The scenario gives no details about pre-release testing. Check the change set, failure logs, and test results to determine whether regression testing missed the affected behavior.

3. Remediation & Regression Protocol:

- **Automated Continuous Regression Testing:** Prior to releasing any maintenance patch or hotfix, the entire baseline regression test suite must execute automatically to verify that existing functionality remains intact.
- **Formal Peer Reviews of Changes:** Mandate formal code inspections for all maintenance edits, evaluating not only the fix itself but also potential side effects on dependent modules.
- **Configuration Management Baselines:** Maintain strict branch tagging; if post-release telemetry detects an abnormal failure intensity spike, implement automated rollback to the previously certified stable baseline.

📝 **Academic Note**

تفسير قفزة الفشل (Failure Spike): صيانة البرمجيات تغير المنطق البرمجي وليست مجرد استبدال قطع ← أي تعديل طفيف قد يولد أخطاء راجعة (Regression faults) في أجزاء كانت سليمة، ما لم يتم إجراء اختبار انحدار شامل (Regression testing) قبل الإطلاق.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.1, pp. 308–310; companion slides 3–5.

Scenario 7.6 · Standards Selection Strategy: ISO 9001 vs SEI CMM Appraisal

A mid-sized software engineering firm (120 developers) currently holds no formal quality certifications. The board of directors debates whether to invest corporate budget into pursuing **ISO 9001 certification** or **SEI CMM Level 3 appraisal**. The firm specializes in developing safety-critical medical device software for hospital networks and military defense contractors.

1. Analyze the strategic suitability, strengths, and limitations of ISO 9001 versus SEI CMM for this specific company.
2. Provide a justified recommendation on which quality framework the organization should prioritize.
3. If the company ultimately decides to pursue both standards, explain the optimal chronological sequence of implementation and how progress in one framework accelerates compliance with the other.

1. Strategic comparison:

- **ISO 9001:** A general quality-management standard. It may matter if a customer or market requires certification, but its broad requirements need interpretation for software work.
- **CMM:** Focuses on software processes and gives more detailed guidance for continuous process improvement. It can be applied to development and maintenance, though it does not directly cover every maintenance-specific concern.

2. Recommendation under a stated assumption: If the board's main objective is to improve the firm's software processes and no current contract requires ISO certification, prioritize CMM-based improvement. If a customer, tender, or applicable requirement explicitly requires ISO 9001 certification, include that requirement in the plan; the scenario does not provide enough information to rank certification above the process-improvement objective.

3. If pursuing both:

- First, identify any customer or market certification requirements and the target audit scope.
- Use CMM to guide software-process improvement where that is the chosen objective.
- Conduct a separate ISO 9001 gap assessment. CMM practices may support parts of the quality system, but the relationship is not one-to-one and a CMM maturity level does not automatically establish ISO 9001 conformity.
- Schedule certification only after the organization has verified and implemented the applicable ISO requirements.

Neither framework by itself should be treated as proof that a particular software product is safe or reliable.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.3–§7.5.3, pp. 353–357; companion slides 103–107.

ASE CHAPTER 7

T7 Software Reliability Calculations Pack

Problem 7.1 · Basic Execution Time Model — Current Failure Intensity Evaluation

Assume that the initial failure intensity is $\lambda_0 = 10 \text{ failures/CPU hr}$ and the total number of failures expected over infinite time is $v_0 = 100 \text{ failures}$.

1. Calculate the failure intensity after 10 failures have been experienced.

2. Calculate the failure intensity after 50 failures have been experienced.

1. **Given:**
   - Initial failure intensity: $\lambda_0 = 10 \text{ failures/CPU hr}$
   - Total expected failures: $v_0 = 100 \text{ failures}$
   - Failure counts experienced: $\mu_1 = 10 \text{ failures}$, $\mu_2 = 50 \text{ failures}$
2. **Required:**
   - Failure intensity $\lambda(10)$ in failures/CPU hr.
   - Failure intensity $\lambda(50)$ in failures/CPU hr.
3. **Model & Assumptions:**
   - **Musa Basic Execution Time Model.**
   - The model assumes a finite expected failure count and a linear reduction in failure intensity as failures are experienced.
4. **Equation:**

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

5. **Substitution & Working:**
   - For $\mu_1 = 10 \text{ failures}$:

$$\lambda(10) = 10 \times \left(1 - \frac{10}{100}\right) = 10 \times (1 - 0.10) = 10 \times 0.90 = 9.0 \text{ failures/CPU hr}$$

   - For $\mu_2 = 50 \text{ failures}$:

$$\lambda(50) = 10 \times \left(1 - \frac{50}{100}\right) = 10 \times (1 - 0.50) = 10 \times 0.50 = 5.0 \text{ failures/CPU hr}$$

6. **Result:**
   - $\lambda(10) = 9.0 \text{ failures/CPU hr}$
   - $\lambda(50) = 5.0 \text{ failures/CPU hr}$
7. **Check / Interpretation:** When $\mu = 0$, $\lambda = 10 \text{ failures/CPU hr}$ (λ_0). When half the faults are removed ($\mu = 50 = v_0/2$), the failure intensity drops by exactly half to $5.0 \text{ failures/CPU hr}$, verifying the strictly linear property of the Basic Execution Time Model.

Source: Aggarwal & Singh (3rd ed.), §7.3.1, Example 7.1, p. 334; companion slides 47–54.

Problem 7.2 · Failure Intensity from Decay Parameter (Exercise 7.16)

Assume that the initial failure intensity is $\lambda_0 = 6 \text{ failures/CPU hr}$. The failure intensity decay parameter is $\theta = 0.02/\text{failure}$. We assume that 45 failures have been experienced. Calculate the current failure intensity.

1. **Given:**
   - Initial failure intensity: $\lambda_0 = 6 \text{ failures/CPU hr}$

- Failure intensity decay parameter: $\theta = 0.02/\text{failure}$
- Failures experienced: $\mu = 45\ \text{failures}$

2. **Required:**
   - Current failure intensity $\lambda(45)$ in failures/CPU hr.
3. **Model & Assumptions:**
   - **Logarithmic Poisson Execution Time Model (Musa–Okumoto).**
     - The decay parameter θ defines the exponential reduction in failure intensity per failure experienced.
4. **Equation (Logarithmic Poisson Model):**

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

5. **Substitution & Working:**

$$\lambda(45) = 6 \times \exp(-0.02 \times 45) = 6 \times \exp(-0.90)$$

Since $\exp(-0.90) \approx 0.40656966$:

$$\lambda(45) = 6 \times 0.40656966 \approx 2.4394\ \text{failures/CPU hr}$$

6. **Result:**
   - $\lambda(45) \approx 2.44\ \text{failures/CPU hr}$.
7. **Check / Interpretation:**
   - Because earlier failures have a greater impact on reducing failure intensity in the logarithmic model than later ones, 45 failures reduce intensity from 6 down to ≈ 2.44, which represents a $\approx 59.3\%$ reduction.

Source: Aggarwal & Singh (3rd ed.), Exercise 7.16, p. 363; companion slide 122.

Problem 7.3 · Multi-Stage Basic Execution Time Evaluation (Exercise 7.18)

Assume that a program will experience 150 failures in infinite time. It has now experienced 80 failures. The initial failure intensity was 10 failures/CPU hr.

1. Determine the current failure intensity.

2. Calculate the failures experienced and failure intensity after 25 and 40 CPU hrs of execution from time zero.

3. Compute additional failures and additional execution time required to reach the failure intensity objective of 2 failures/CPU hr from the current state.

1. **Given:**
   - Total failures over infinite time: $v_0 = 150\ \text{failures}$
   - Initial failure intensity: $\lambda_0 = 10\ \text{failures/CPU hr}$
   - Failures experienced currently: $\mu_P = 80\ \text{failures}$
   - Evaluation execution times: $\tau_1 = 25\ \text{CPU hrs}$, $\tau_2 = 40\ \text{CPU hrs}$
   - Target failure intensity objective: $\lambda_F = 2\ \text{failures/CPU hr}$
2. **Required:**
   - (i) Current failure intensity λ_P.
   - (ii) Failures experienced $\mu(25)$, $\mu(40)$ and failure intensity $\lambda(25)$, $\lambda(40)$.
   - (iii) Additional failures $\Delta\mu$ and additional execution time $\Delta\tau$ to reach $\lambda_F = 2\ \text{failures/CPU hr}$.
3. **Model & Assumptions:**
   - **Musa Basic Execution Time Model.**
   - All relations express failures experienced μ and failure intensity λ as functions of execution time τ.
4. **Equations:**
   - Current intensity from experienced failures:

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

- Intensity and failures as functions of execution time τ:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

$$\mu(\tau) = v_0 \left[1 - \exp\left(-\frac{\lambda_0}{v_0}\tau\right)\right]$$

- Increments between present state (λ_P) and target objective (λ_F):

$$\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$$

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$$

5. **Substitution & Working:**

- **Part (i): Current failure intensity:**

$$\lambda_P = 10 \times \left(1 - \frac{80}{150}\right) = 10 \times \left(1 - \frac{8}{15}\right) = 10 \times \frac{7}{15} = \frac{70}{15} \approx 4.6667 \approx 4.67 \text{ failures/CPU hr}$$

- **Part (ii): Values at $\tau = 25$ and $\tau = 40$ CPU hrs:** Ratio $\frac{\lambda_0}{v_0} = \frac{10}{150} = \frac{1}{15} \approx 0.066667 \text{ hr}^{-1}$.
  - At $\tau = 25$ CPU hrs:

$$\frac{\lambda_0}{v_0}\tau = \frac{25}{15} = \frac{5}{3} \approx 1.66667$$

$$\lambda(25) = 10 \times \exp\left(-\frac{5}{3}\right) \approx 10 \times 0.188876 = 1.8888 \approx 1.89 \text{ failures/CPU hr}$$

$$\mu(25) = 150 \times \left[1 - \exp\left(-\frac{5}{3}\right)\right] = 150 \times (1 - 0.188876) = 150 \times 0.811124 = 121.669 \approx 121.67 \text{ failures}$$

  - At $\tau = 40$ CPU hrs:

$$\frac{\lambda_0}{v_0}\tau = \frac{40}{15} = \frac{8}{3} \approx 2.66667$$

$$\lambda(40) = 10 \times \exp\left(-\frac{8}{3}\right) \approx 10 \times 0.069483 = 0.6948 \approx 0.69 \text{ failures/CPU hr}$$

$$\mu(40) = 150 \times \left[1 - \exp\left(-\frac{8}{3}\right)\right] = 150 \times (1 - 0.069483) = 150 \times 0.930517 = 139.578 \approx 139.58 \text{ failures}$$

- **Part (iii): Additional failures and execution time to reach $\lambda_F = 2$ failures/CPU hr:**
  - Using current state $\lambda_P = \frac{70}{15} \approx 4.6667$ failures/CPU hr and $\mu_P = 80$:

$$\Delta\mu = \frac{150}{10} \times \left(\frac{70}{15} - 2\right) = 15 \times \left(\frac{40}{15}\right) = 40 \text{ failures}$$

  (Direct cross-check: Target failures $\mu_F = 150 \times (1 - 2/10) = 150 \times 0.8 = 120$; $\Delta\mu = 120 - 80 = 40$ failures).
  - **Additional execution time:**
    $\Delta\tau = \frac{150}{10} \times \ln\left(\frac{4.6667}{2}\right) = 15 \times \ln(2.33333) = 15 \times 0.847298 \approx 12.7095 \approx 12.71$ CPU hrs

6. **Result:**

- (i) $\lambda_P \approx 4.67$ failures/CPU hr
- (ii) At $\tau = 25$: $\lambda(25) \approx 1.89$ failures/CPU hr, $\mu(25) \approx 121.67$ failures;
  At $\tau = 40$: $\lambda(40) \approx 0.69$ failures/CPU hr, $\mu(40) \approx 139.58$ failures.
- (iii) $\Delta\mu = 40$ failures, $\Delta\tau \approx 12.71$ CPU hrs.

7. Check / Interpretation: Target intensity $2\ \text{failures/CPU hr}$ is reached at total execution time $\tau_F = 15\ln(10/2) \approx 24.14$ CPU hours. The current state ($\mu_P = 80$, $\lambda_P = 70/15$) occurs at $\tau_P = 15\ln(10/(70/15)) \approx 11.43$ CPU hours, so the additional time is $\Delta\tau = \tau_F - \tau_P \approx 12.71$ CPU hours, consistent with the result above.

Source: Aggarwal & Singh (3rd ed.), Exercise 7.18, p. 363; companion slide 123.

Problem 7.4 · Release Objective Evaluation with Unit Conversion (Exercise 7.24)

A program is expected to have 250 faults. It is also assumed that one fault may lead to one failure. The initial failure intensity is 5 failures/CPU hr. The program is released with a failure intensity objective of 4 failures per 10 CPU hours. Calculate the number of failures experienced before release.

1. **Given:**
   - Total faults / failures over infinite time: $v_0 = 250$ failures (1 fault = 1 failure assumption).
   - Initial failure intensity: $\lambda_0 = 5\ \text{failures/CPU hr}$.
   - Release objective: 4 failures per 10 CPU hours.
2. **Required:**
   - Number of failures experienced before release (μ_F).
3. **Model & Assumptions:**
   - **Musa Basic Execution Time Model.**
   - Units must be strictly normalized to failures/CPU hr before substitution.
4. **Unit Normalization & Equation:**

$$\lambda_F = \frac{4\ \text{failures}}{10\ \text{CPU hrs}} = 0.40\ \text{failures/CPU hr}$$

$$\lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{v_0}\right) \implies \mu = v_0\left(1 - \frac{\lambda_F}{\lambda_0}\right)$$

5. **Substitution & Working:**

$$\mu_F = 250 \times \left(1 - \frac{0.40}{5.0}\right) = 250 \times (1 - 0.08) = 250 \times 0.92 = 230\ \text{failures}$$

6. **Result:**
   - $\mu_F = 230\ \text{failures experienced before release.}$
7. **Check / Interpretation:**
   - The remaining unmanifested faults at release are $v_0 - \mu_F = 250 - 230 = 20\ \text{faults}$.
   - The fraction of faults removed is $230/250 = 92\%$, which corresponds to an intensity reduction from 5.0 to 0.40 (92% reduction).

Source: Aggarwal & Singh (3rd ed.), Exercise 7.24, p. 364; companion slide 125.

Problem 7.5 · Logarithmic Poisson Model — Failures Experienced after 20 CPU Hours

Assume that the initial failure intensity is $\lambda_0 = 20\ \text{failures/CPU hr}$ and the failure intensity decay parameter is $\theta = 0.02/\text{failure}$. Determine the number of failures experienced after 20 CPU hours of execution.

1. **Given:**
   - Initial failure intensity: $\lambda_0 = 20\ \text{failures/CPU hr}$
   - Failure intensity decay parameter: $\theta = 0.02/\text{failure}$
   - Execution time: $\tau = 20\ \text{CPU hrs}$
2. **Required:**
   - Expected number of failures experienced $\mu(20)$.
3. **Model & Assumptions:**

- **Musa–Okumoto Logarithmic Poisson Execution Time Model.**
- The failure intensity decreases exponentially with failures experienced.

4. **Equation:**

$$\mu(\tau) = \frac{1}{\theta}\ln(\lambda_0\theta\tau + 1)$$

5. **Substitution & Working:**
   - Inner dimensionless term:

$$\lambda_0\theta\tau + 1 = 20 \times 0.02 \times 20 + 1 = 8 + 1 = 9$$

   - Logarithm evaluation:

$$\ln(9) \approx 2.19722458$$

   - Multiplication:

$$\mu(20) = \frac{1}{0.02} \times 2.19722458 = 50 \times 2.19722458 = 109.8612 \approx 109.86 \text{ failures}$$

6. **Result:**
   - $\mu(20) = 109.86 \text{ failures} \approx 110 \text{ failures}$.
   - The displayed worked integer is 109; direct substitution into the stated equation gives 109.86.
7. **Check / Interpretation:**
   - The result follows from $\lambda_0\theta\tau + 1 = 9$ and is consistent with the logarithmic-model equation.

Source: Aggarwal & Singh (3rd ed.), §7.3.2, Example 7.2, pp. 336–337; companion slide 62.

Problem 7.6 · Logarithmic Poisson Model — Failure Intensity at Specific Execution Time

Using the parameters of Problem 7.5 ($\lambda_0 = 20 \text{ failures/CPU hr}$, $\theta = 0.02/\text{failure}$), calculate the failure intensity after 20 CPU hours of execution.

1. **Given:**
   - Initial failure intensity: $\lambda_0 = 20 \text{ failures/CPU hr}$
   - Failure intensity decay parameter: $\theta = 0.02/\text{failure}$
   - Execution time: $\tau = 20 \text{ CPU hrs}$
2. **Required:**
   - Failure intensity $\lambda(20)$ in failures/CPU hr.
3. **Model & Assumptions:**
   - **Musa–Okumoto Logarithmic Poisson Execution Time Model.**
4. **Equations:**
   - Method 1 (Direct function of execution time τ):

$$\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}$$

   - Method 2 (Function of failures experienced μ):

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

5. **Substitution & Working:**
   - **Method 1 (Direct from $\tau = 20$):**

$$\lambda(20) = \frac{20}{20 \times 0.02 \times 20 + 1} = \frac{20}{9} \approx 2.2222 \text{ failures/CPU hr}$$

- **Method 2 (using the unrounded $\mu(20) = 109.8612$ from Problem 7.5):**

$$\lambda(20) = 20\exp(-0.02 \times 109.8612) \approx 2.2222 \text{ failures/CPU hr}$$

6. **Result:**
   - $\lambda(20) \approx 2.22 \text{ failures/CPU hr}$.
7. **Check / Interpretation:**
   - The two equivalent forms agree: $\lambda(20) = \lambda_0/(\lambda_0\theta\tau + 1) = \lambda_0 e^{-\theta\mu(20)}$.

Source: Aggarwal & Singh (3rd ed.), §7.3.2, Example 7.2, p. 336; companion slide 62.

Problem 7.7 · Logarithmic Model Execution Progress at 25 and 50 CPU Hours (Exercise 7.20)

Assume that the initial failure intensity is $\lambda_0 = 10 \text{ failures/CPU hr}$ and the failure intensity decay parameter is $\theta = 0.03/\text{failure}$. We have experienced 75 failures up to this time. Find the failures experienced and failure intensity after 25 and 50 CPU hours of execution.

1. **Given:**
   - Initial failure intensity: $\lambda_0 = 10 \text{ failures/CPU hr}$
   - Failure intensity decay parameter: $\theta = 0.03/\text{failure}$
   - Failures experienced at the stated current point: $\mu_P = 75 \text{ failures}$
   - Requested intervals: $\Delta\tau = 25$ and 50 CPU hours
2. **Required:**
   - Failures experienced and failure intensity at each requested point.
3. **Model & Assumptions:**
   - **Logarithmic Poisson Execution Time Model.** The main calculation treats 25 and 50 CPU hours as additional execution time from the stated current point. The alternate cumulative-time reading is also shown because the wording does not specify the reference point.
4. **Equations:**

$$\lambda_P = \lambda_0 e^{-\theta\mu_P}, \qquad \mu(\tau_P + \Delta\tau) = \mu_P + \frac{1}{\theta}\ln(1 + \theta\lambda_P\Delta\tau)$$

$$\lambda(\tau_P + \Delta\tau) = \frac{\lambda_P}{1 + \theta\lambda_P\Delta\tau}, \qquad \tau_P = \frac{e^{\theta\mu_P} - 1}{\lambda_0\theta}$$

   - If the requested times are cumulative from the start: $\mu(\tau) = \frac{1}{\theta}\ln(\lambda_0\theta\tau + 1)$ and $\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau+1}$.
5. **Substitution & Working:**
   - Current state: $\lambda_P = 10e^{-0.03(75)} \approx 1.05399$ failures/CPU hr and $\tau_P = (e^{2.25} - 1)/0.30 \approx 28.29$ CPU hours.
   - **After an additional 25 CPU hours:** $\mu = 75 + \frac{1}{0.03}\ln(1 + 0.03(1.05399)(25)) \approx 94.42$ failures; $\lambda = \frac{1.05399}{1+0.03(1.05399)(25)} \approx 0.589$ failures/CPU hr.
   - **After an additional 50 CPU hours:** $\mu = 75 + \frac{1}{0.03}\ln(1 + 0.03(1.05399)(50)) \approx 106.61$ failures; $\lambda = \frac{1.05399}{1+0.03(1.05399)(50)} \approx 0.408$ failures/CPU hr.
   - **If 25 and 50 hours mean cumulative time from the start:** $\mu(25) \approx 71.34$, $\lambda(25) \approx 1.176$; $\mu(50) \approx 92.42$, $\lambda(50) = 0.625$. However, the stated 75 failures correspond to $\tau_P \approx 28.29$ hours, so the 25-hour point precedes the stated current state.
6. **Result:**
   - Main reading (25 and 50 additional CPU hours): $(\mu, \lambda) \approx (94.42, 0.589)$ and $(106.61, 0.408)$, respectively.
   - Cumulative-from-start reading: $(\mu(25), \lambda(25)) \approx (71.34, 1.176)$ and $(\mu(50), \lambda(50)) \approx (92.42, 0.625)$; this conflicts with the stated current point at 75 failures for $\tau = 25$.
7. **Check / Interpretation:**
   - Under the main reading, both requested times follow the present state and failure intensity decreases as execution time increases.

Source: Aggarwal & Singh (3rd ed.), Exercise 7.20, p. 363; companion slide 123.

Problem 7.8 · Comparative Calculation: Basic vs Logarithmic Poisson Model (Exercise 7.21)

The following parameters for the Basic Execution Time Model and the Logarithmic Poisson Execution Time Model are given:

- **Basic Model:** $\lambda_0 = 5 \text{ failures/CPU hr}$, $v_0 = 125 \text{ failures}$
- **Logarithmic Poisson Model:** $\lambda_0 = 25 \text{ failures/CPU hr}$, $\theta = 0.30/\text{failure}$

Determine the additional failures ($\Delta\mu$) and additional execution time ($\Delta\tau$) required to reach the failure intensity objective of $\lambda_F = 0.10 \text{ failure/CPU hr}$ for both models (from their initial states λ_0).

1. **Given:**
   - Basic Model: $\lambda_{0,B} = 5 \text{ failures/CPU hr}$, $v_0 = 125 \text{ failures}$
   - Logarithmic Model: $\lambda_{0,L} = 25 \text{ failures/CPU hr}$, $\theta = 0.30/\text{failure}$
   - Common target failure intensity: $\lambda_F = 0.10 \text{ failure/CPU hr}$
2. **Required:**
   - For Basic Model: $\Delta\mu_B$ and $\Delta\tau_B$.
   - For Logarithmic Model: $\Delta\mu_L$ and $\Delta\tau_L$.
3. **Model & Assumptions:**
   - Comparison evaluated from initial state ($\lambda_P = \lambda_0$, $\mu_P = 0$) to target release objective (λ_F).
4. **Equations:**
   - **Basic Execution Time Model:**

$$\Delta\mu_B = \frac{v_0}{\lambda_{0,B}}(\lambda_{0,B} - \lambda_F) = v_0\left(1 - \frac{\lambda_F}{\lambda_{0,B}}\right)$$

$$\Delta\tau_B = \frac{v_0}{\lambda_{0,B}} \ln\left(\frac{\lambda_{0,B}}{\lambda_F}\right)$$

   - **Logarithmic Poisson Model:**

$$\Delta\mu_L = \frac{1}{\theta} \ln\left(\frac{\lambda_{0,L}}{\lambda_F}\right)$$

$$\Delta\tau_L = \frac{1}{\theta}\left(\frac{1}{\lambda_F} - \frac{1}{\lambda_{0,L}}\right)$$

5. **Substitution & Working:**
   - **Case 1: Basic Model Calculations:**
     - Scaling factor: $\frac{v_0}{\lambda_{0,B}} = \frac{125}{5} = 25 \text{ CPU hrs}$.
     - Additional failures:

$$\Delta\mu_B = 125 \times \left(1 - \frac{0.10}{5}\right) = 125 \times (1 - 0.02) = 125 \times 0.98 = 122.5 \text{ failures}$$

     - **Additional execution time:**
       $\Delta\tau_B = 25 \times \ln\left(\frac{5}{0.10}\right) = 25 \times \ln(50) = 25 \times 3.912023 \approx 97.8006 \approx 97.80 \text{ CPU hrs}$
   - **Case 2: Logarithmic Poisson Model Calculations:**
     - Scaling factor: $\frac{1}{\theta} = \frac{1}{0.30} = \frac{10}{3} \approx 3.3333$.
     - Ratio: $\frac{\lambda_{0,L}}{\lambda_F} = \frac{25}{0.10} = 250$.
     - Additional failures:

$$\Delta\mu_L = \frac{10}{3} \times \ln(250) = \frac{10}{3} \times 5.521461 = 18.40487 \approx 18.40 \text{ failures}$$

     - **Additional execution time:** $\Delta\tau_L = \frac{1}{0.30} \times \left(\frac{1}{0.10} - \frac{1}{25}\right) = \frac{10}{3} \times (10 - 0.04) = \frac{10}{3} \times 9.96 = 33.20 \text{ CPU hrs}$
6. **Result:**
   - **Basic Model:** $\Delta\mu_B = 122.5 \text{ failures}$, $\Delta\tau_B \approx 97.80 \text{ CPU hrs}$.

- **Logarithmic Model:** $\Delta\mu_L \approx 18.40$ failures, $\Delta\tau_L = 33.20$ CPU hrs.

7. **Check / Interpretation:**
   - Although the Logarithmic Poisson model starts at a higher initial intensity ($\lambda_0 = 25$ vs 5), its high decay parameter ($\theta = 0.30/\text{failure}$) drops failure intensity far more rapidly per observed failure, requiring only ≈ 18.4 failures and 33.2 CPU hours compared to 122.5 failures and 97.8 CPU hours under the Basic model.

Source: Aggarwal & Singh (3rd ed.), Exercise 7.21, p. 364; companion slide 124.

Problem 7.9 · Calendar Time Component — Resource-Limited Execution Time Ratio (Examples 7.4–7.6)

During a software testing phase, resource usage is modeled as $\chi_k = \theta_k \tau + \mu_k \mu$. For the failure identification personnel resource ($k = I$):

- $\theta_I = 15$ person-hrs/CPU hr
- $\mu_I = 2$ person-hrs/failure
- Available personnel: $P_I = 2$ persons
- Utilization factor: $\rho_I = 0.80$ If an interval requires an additional execution time of $\Delta\tau = 10$ CPU hrs and experiences $\Delta\mu = 25$ failures, calculate:
- The total resource usage $\Delta\chi_I$ in person-hours.
- The calendar time Δt required to complete this testing phase (in calendar hours and 8-hour work days).

1. Given:

- Execution time increment: $\Delta\tau = 10$ CPU hrs
- Failure increment: $\Delta\mu = 25$ failures
- Effort parameters: $\theta_I = 15$ person-hrs/CPU hr, $\mu_I = 2$ person-hrs/failure
- Staffing and utilization: $P_I = 2$, $\rho_I = 0.80$

2. Required:

- Total resource requirement $\Delta\chi_I$ in person-hours.
- Calendar time Δt in calendar hours and working days.

3. Model & Assumptions:

- **Musa Calendar Time Component Model.**
- Testing is conducted in the Failure Identification Personnel Limited segment.
- 1 working day equals 8 productive calendar hours.

4. Equations:

$$\Delta\chi_I = \theta_I \Delta\tau + \mu_I \Delta\mu$$

$$\Delta t = \frac{\Delta\chi_I}{P_I \rho_I}$$

5. Substitution & Working:

- **Resource Usage:**

$$\Delta\chi_I = (15 \times 10) + (2 \times 25) = 150 + 50 = 200 \text{ person-hrs}$$

- **Calendar Time:**

$$\Delta t = \frac{200}{2 \times 0.80} = \frac{200}{1.60} = 125.0 \text{ calendar hrs}$$

- In 8-hour working days:

$$\Delta t_{\text{days}} = \frac{125}{8} = 15.625 \text{ work days} \approx 16 \text{ work days}$$

6. Result:

- $\Delta\chi_I = 200$ person-hrs
- $\Delta t = 125.0$ calendar hrs (15.6 working days)

7. Check / Interpretation:

- The interval-average ratio is $\Delta t/\Delta\tau = 125/10 = 12.5$ calendar hours per CPU hour. This agrees with average resource use of $200/10 = 20$ person-hours per CPU hour divided by available utilized capacity $P_I\rho_I = 1.6$ persons.

Source: Aggarwal & Singh (3rd ed.), §7.3.3, Examples 7.4–7.6, pp. 340–342; companion slides 72–82.

Problem 7.10 · Jelinski–Moranda Model & Bug Seeding Population Estimation (Example 7.7 & Exercise 7.26)

1. **Part A (Jelinski–Moranda Model):** A software system has an estimated initial fault count of $N = 50$, with a fault failure rate constant of $\phi = 0.002/\text{hr}$. Calculate the failure hazard rate $\lambda(t_i)$ when debugging starts ($i = 1$), after 10 faults have been removed ($i = 11$), and after 40 faults have been removed ($i = 41$).
2. **Part B (Harlan Mills Bug Seeding):** An organization seeds $S = 20$ artificial bugs into a software program before testing. Testing discovers a total of 30 bugs, of which $s = 12$ are seeded bugs and $n = 18$ are indigenous bugs. Estimate the total original indigenous bug population $\hat{N}$ and the number of unmanifested indigenous bugs remaining in the code.

Part A: Jelinski–Moranda Evaluation

1. **Given:**
   - Initial fault count: $N = 50$
   - Proportionality constant: $\phi = 0.002/\text{hr}$
   - Indices: $i = 1$ (first failure), $i = 11$ (11th failure / 10 removed), $i = 41$ (41st failure / 40 removed)
2. **Required:**
   - Hazard rate $\lambda(t_1)$, $\lambda(t_{11})$, and $\lambda(t_{41})$ in $\text{failures}/\text{hr}$.
3. **Model & Equation:**

$$\lambda(t_i) = \phi[N - (i-1)]$$

4. **Working:**
   - For $i = 1$:

$$\lambda(t_1) = 0.002 \times [50 - (1-1)] = 0.002 \times 50 = 0.10 \text{ failures}/\text{hr}$$

   - For $i = 11$:

$$\lambda(t_{11}) = 0.002 \times [50 - (11-1)] = 0.002 \times 40 = 0.08 \text{ failures}/\text{hr}$$

   - For $i = 41$:

$$\lambda(t_{41}) = 0.002 \times [50 - (41-1)] = 0.002 \times 10 = 0.02 \text{ failures}/\text{hr}$$

5. **Result (Part A):**
   - $\lambda(t_1) = 0.10 \text{ failures}/\text{hr}$
   - $\lambda(t_{11}) = 0.08 \text{ failures}/\text{hr}$
   - $\lambda(t_{41}) = 0.02 \text{ failures}/\text{hr}$

Part B: Bug Seeding Estimation

1. **Given:**
   - Seeded artificial bugs: $S = 20$
   - Discovered seeded bugs: $s = 12$
   - Discovered indigenous bugs: $n = 18$
2. **Required:**
   - Estimated total indigenous bugs $\hat{N}$.
   - Estimated remaining unmanifested indigenous bugs N_{rem}.
3. **Model & Assumptions:**
   - **Mills Bug Seeding Model (Capture–Recapture Analogy).**
   - Indigenous bugs and seeded bugs have identical detection probabilities and are randomly distributed throughout the code.
4. **Equation:**

$$\frac{s}{S} = \frac{n}{\hat{N}} \implies \hat{N} = \frac{S \times n}{s}$$

$$N_{\text{rem}} = \hat{N} - n$$

5. **Working:**

$$\hat{N} = \frac{20 \times 18}{12} = \frac{360}{12} = 30 \text{ indigenous bugs}$$

$$N_{\text{rem}} = 30 - 18 = 12 \text{ indigenous bugs remaining}$$

6. **Result (Part B):**
   - Estimated total indigenous bugs: $\hat{N} = 30$ bugs.
   - Estimated remaining indigenous bugs: $N_{\text{rem}} = 12$ bugs.
7. **Check / Interpretation:**
   - Recovery ratio for seeded bugs is $12/20 = 60\%$. Under equal detectability, the 18 discovered indigenous bugs also represent 60% of total indigenous bugs ($18/0.60 = 30$). Arithmetic is completely consistent.

Source: Aggarwal & Singh (3rd ed.), §7.3.4 & §7.3.5, Example 7.7, pp. 343–346; companion slides 89–93.
